FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

DESARROLLO DE APLICACIONES MÓVILES

“EQUISHARE: aplicación Android para el alquiler compartido de equipos entre pymes de Trujillo, alineada al ODS 12”

Autores:

ALIAGA HILARIO, Mariano Daniel (100 %)
ASPIROS BERMÚDEZ, Diego Sebastián (100 %)
CHUQUIZUTA RISCO, Erick Felipe (100 %)
MUÑOZ CLAVIJO, Brayan Alexander (100 %)
SABROSO ABANTO, Eduardo Wilfredo (100 %)
TERRONES MIRANDA, Alexander Joel (100 %)
TORRES CALLE, Rick Roy (100 %)

Docente del Curso:

TUMBA BURGOS, Victor Piero Milcar

Trujillo – Perú
2026-2

TABLA DE CONTENIDOS

I. RESUMEN....5
II. INTRODUCCIÓN....7
2.1. Descripción de la problemática....7
2.2. Descripción del proyecto....8
III. ANÁLISIS DEL PROBLEMA....9
3.1. Antecedentes....9
3.1.1. Antecedentes académicos....9
3.1.2. Casos de la industria....10
3.1.3. Tendencias de las aplicaciones móviles Android....11
3.2. Identificación y formulación del problema....12
3.3. Propuestas....13
3.3.1. Propuesta 01: aplicación Android nativa con API REST propia y Firebase (seleccionada)....13
3.3.2. Propuesta 02: aplicación web progresiva (PWA) adaptable....13
3.3.3. Propuesta 03: aplicación Android solo con Firebase como backend....14
3.4. Definición de objetivos....14
3.4.1. Objetivo general....14
3.4.2. Objetivos específicos....14
IV. HERRAMIENTAS DE INGENIERÍA....16
4.1. Comparación y selección de metodologías....16
4.2. Estándares de ingeniería aplicados....17
4.3. Selección de herramientas de hardware y software....17
4.4. Recolección, análisis y procesamiento de datos....18
4.5. Requisitos del usuario final y del servidor....19
V. GENERACIÓN DE SOLUCIONES....20
5.1. Evaluación de tecnologías de aplicaciones móviles....20
5.2. Propuesta de solución y restricciones realistas....21
5.3. Investigación 1: marketing irresponsable en aplicaciones y propuestas éticas. 21
VI. METODOLOGÍA DE DESARROLLO A UTILIZAR....23
6.1. Aplicación móvil....23
6.1.1. Planificación: EDT y cronograma....23
6.1.2. Análisis: Product Backlog, reglas de negocio y requisitos....25
6.1.3. Diseño: prototipos, flujo de pantallas y arquitectura....28
6.1.4. Desarrollo: resultados por sprint....33
6.1.5. Pruebas....36

6.1.6. Implementación....38
6.1.7. Protección de datos personales....39
6.2. Evidencias del funcionamiento....39
6.3. Repositorios....40
VII. COMPLEMENTO....41
7.1. Evidencia de participación....41
7.2. Video de presentación del proyecto (link)....41
7.3. PPT de sustentación....41
7.4. Link del repositorio donde se encuentran los proyectos (código)....41
VIII. RESULTADOS....42
8.1. Resultados en base a los objetivos del proyecto....42
8.2. Discusión de resultados....42
IX. CONCLUSIONES....44
X. REFERENCIAS O BIBLIOGRAFÍA....46
XI. ANEXOS....51
Anexo 1. Investigación 1: marketing irresponsable en aplicaciones y propuestas éticas....51
Anexo 2. Historias de usuario con criterios de aceptación....52

ÍNDICE DE TABLAS

Tabla 1. Problema general y problemas específicos....12
Tabla 2. Comparación de metodologías de desarrollo....16
Tabla 3. Estándares y guías aplicados en el proyecto....17
Tabla 4. Herramientas de software seleccionadas....17
Tabla 5. Requisitos del usuario final y del servidor....19
Tabla 6. Ventajas y desventajas de las tecnologías evaluadas....20
Tabla 7. Restricciones realistas y decisiones de diseño....21
Tabla 8. Plan de sprints....25
Tabla 9. Product Backlog priorizado....26
Tabla 10. Reglas de negocio e implementación....27
Tabla 11. Requisitos no funcionales....28
Tabla 12. Endpoints de la API REST....33
Tabla 13. Casos de prueba funcionales....37
Tabla 14. Resultados por objetivo específico....42
Tabla 15. Casos documentados de marketing irresponsable....51
Tabla 16. Principios de diseño ético e implementación en EQUISHARE....52

Tabla 17. Criterios de aceptación de las historias terminadas.....52

ÍNDICE DE FIGURAS

Figura 1. Diagrama de Ishikawa del problema de subutilización de equipos.....12
Figura 2. Estructura de desglose del trabajo (EDT) de EQUISHARE.....24
Figura 3. Cronograma de sprints del proyecto.....25
Figura 4. Prototipos de alta fidelidad de las pantallas principales.....29
Figura 5. Flujo de pantallas y navegación de la aplicación.....30
Figura 6. Arquitectura general de la solución.....31
Figura 7. Patrón MVVM aplicado a la pantalla de catálogo.....31
Figura 8. Modelo de datos del servidor.....32
Figura 9. Modelo de datos local (Room y DataStore) y documentos de Cloud Firestore 32
Figura 10. Resultado de las pruebas automatizadas de la API REST.....36
Figura 11. Resultado de las pruebas unitarias de la capa de dominio de la app.....37
Figura 12. Consumo de la API REST con el servidor en funcionamiento.....38
Figura 13. Vista general de las diapositivas de sustentación.....41

ÍNDICE DE ANEXOS

Anexo 1. Investigación 1: marketing irresponsable en aplicaciones y propuestas éticas 51
Anexo 2. Historias de usuario con criterios de aceptación.....52

I. RESUMEN

Las pequeñas y medianas empresas de construcción, organización de eventos y servicios técnicos de Trujillo suelen comprar maquinaria y equipos que usan solo de forma esporádica, mientras otras empresas los necesitan por pocos días. Para atender esta subutilización se desarrolló EQUISHARE, una aplicación móvil Android nativa que conecta a empresas con equipos ociosos con empresas que desean alquilarlos de forma temporal, en línea con el Objetivo de Desarrollo Sostenible 12 (producción y consumo responsables). El proyecto siguió la metodología Scrum y se construyó con Kotlin, Jetpack Compose y arquitectura MVVM, persistencia local con Room y DataStore, una API REST propia en Node.js y Express, Firebase y Google Maps. Como resultados, la API ofrece 21 operaciones documentadas en OpenAPI 3.0, aplica las ocho reglas de negocio del producto y aprueba sus 21 pruebas automatizadas; la aplicación cuenta con 11 pantallas que gestionan los estados de carga, error, vacío y sin conexión, y 16 pruebas unitarias aprobadas en la capa de dominio. El diseño incorpora principios de marketing ético y de protección de datos personales conforme a la Ley N.° 29733.

Palabras clave: aplicación móvil Android, economía colaborativa B2B, alquiler de equipos, arquitectura MVVM, ODS 12.

Abstract

Small and medium-sized construction, event-organization and technical-service companies in Trujillo often buy machinery and equipment that they use only occasionally, while other companies need them for only a few days. To address this underuse, EQUISHARE was developed: a native Android mobile application that connects companies with idle equipment to companies that want to rent it temporarily, in line with Sustainable Development Goal 12 (responsible consumption and production). The project followed the Scrum methodology and was built with Kotlin, Jetpack Compose and MVVM architecture, local persistence with Room and DataStore, a custom REST API in Node.js and Express, Firebase and Google Maps. As for the results, the API offers 21 operations documented in OpenAPI 3.0, applies the eight business rules of the product and passes its 21 automated tests; the app has 11 screens that handle loading, error, empty and

offline states, and 16 passing unit tests in the domain layer. The design incorporates ethical marketing principles and personal data protection in accordance with Law No. 29733.

Keywords: Android mobile application, B2B sharing economy, equipment rental, MVVM architecture, SDG 12.

II. INTRODUCCIÓN

2.1. Descripción de la problemática

El tejido empresarial peruano está formado casi por completo por micro y pequeñas empresas: según el Ministerio de la Producción, el 99,1 % de las 2 346 592 empresas formales que cerró el 2024 son mypes, y La Libertad concentra el 5,3 % del total (123 162 empresas) (Agencia Andina, 2025). Entre ellas, las que ejecutan obras, organizan eventos o brindan servicios técnicos necesitan equipos como mezcladoras, andamios, generadores o proyectores, pero los usan de manera intermitente, según el calendario de cada proyecto.

Comprar un equipo que solo se usa algunos días al mes inmoviliza capital y genera costos de mantenimiento, almacenamiento y depreciación. La literatura sobre economía colaborativa entre empresas describe esta situación: compañías con activos ociosos que podrían generar ingresos y compañías que solo requieren acceso temporal, conectadas por plataformas digitales (Benoit et al., 2025). En la maquinaria de construcción, los desarrolladores de la plataforma Sharemac señalan que las empresas usan sus máquinas cerca de la mitad del tiempo y que las grúas permanecen sin uso cerca del 75 % del tiempo en obra (Constructor University, 2019).

En Trujillo, a partir de la observación del mercado y de conversaciones informales con empresas del rubro, el equipo identificó que el alquiler entre pymes se resuelve por llamadas y contactos personales, sin registro de disponibilidad, sin contratos estandarizados y sin mecanismos que generen confianza. Este diagnóstico es preliminar y se validará con una encuesta. La confianza es un factor clave en la economía colaborativa, pues depende de la reputación, de la confianza en la plataforma y de la experiencia de interacción (ter Huurne et al., 2017).

El canal móvil es el más adecuado: Android concentra el 85,64 % de los sistemas operativos móviles del Perú (StatCounter, 2026) y el 90,2 % de las personas que usan internet accede desde el celular (INEI, 2026). Además, el problema se relaciona con el ODS 12, en particular con las metas 12.2 (uso eficiente de recursos) y 12.5 (reducción de desechos mediante reutilización): compartir un equipo existente en lugar de comprar uno nuevo prolonga su vida

útil (United Nations, s. f.). Como advierten Curtis y Mont (2020), la economía colaborativa no es sostenible por defecto, por lo que EQUISHARE se diseñó para cumplir las condiciones que estos autores identifican.

2.2. Descripción del proyecto

EQUISHARE es una plataforma móvil B2B de alquiler y préstamo de equipos entre empresas, con Trujillo como mercado piloto. La empresa arrendadora publica sus equipos con foto, precio por día y ubicación; la empresa arrendataria los busca por categoría, texto o cercanía, revisa la disponibilidad real y las calificaciones, y reserva un rango de fechas conociendo de antemano el costo total, la comisión de la plataforma y la política de cancelación. Tras la devolución, ambas empresas se califican mutuamente para construir una reputación verificable.

La solución tiene dos componentes: una aplicación Android nativa (Kotlin, Jetpack Compose y MVVM) con persistencia local para funcionar sin conexión, y una API REST en Node.js y Express que concentra las reglas de negocio. Se apoya en Firebase para la autenticación y los datos del usuario, y en Google Maps para la geolocalización. El detalle técnico se desarrolla en el capítulo VI.

La versión desarrollada cubre el ciclo completo del alquiler: registro de la empresa, publicación, búsqueda, reserva, entrega, devolución y calificación. Quedan como trabajo futuro la validación de fotos con Machine Learning, las notificaciones push, la pasarela de pagos simulada, el panel administrativo y la verificación del RUC con el padrón de SUNAT. No forman parte del alcance la versión iOS, la logística de transporte, la integración con sistemas ERP ni la expansión fuera de Trujillo.

III. ANÁLISIS DEL PROBLEMA

3.1. Antecedentes

Los antecedentes se organizan en estudios académicos sobre plataformas de compartición de equipos, casos de la industria y tendencias del desarrollo de aplicaciones Android. Para cada uno se indica su aporte a la propuesta.

3.1.1. Antecedentes académicos

Antecedente 1. Benoit et al. (2025) proponen en el *Journal of Business Research* un marco conceptual de la economía colaborativa entre empresas (B2B sharing economy). El modelo requiere tres actores: una plataforma digital especializada, empresas dispuestas a compartir activos ociosos y empresas dispuestas a pagar por el acceso temporal; para el dueño del activo, compartir suele ser un servicio complementario y no su negocio principal. El estudio ofrece como ejemplo a una constructora que alquila equipos sin uso entre proyectos. Este marco sustenta el modelo de EQUISHARE, que reconoce dos roles por empresa (arrendadora y arrendataria) dentro de la misma cuenta y deja la comisión del 12 % a cargo de quien obtiene el ingreso adicional.

Antecedente 2. Goswami y Arputham (2026) formulan un marco conceptual para una plataforma de dos lados que conecte a pequeños contratistas que necesitan equipos de construcción con grandes contratistas y distribuidores que los poseen, inspirado en las plataformas de transporte compartido. El marco incluye palancas operativas (rapidez, precio, disponibilidad e idoneidad) y dimensiones de gestión de información (calidad, precisión, simetría y continuidad de la información compartida). La propuesta adopta esas dimensiones mediante la disponibilidad real por fechas, la cotización transparente y la información verificable del equipo, aunque los autores la presentan como base teórica que aún requiere validación empírica.

Antecedente 3. Rathnayake et al. (2024), en las actas de la conferencia ARCOM, combinan una revisión sistemática con el análisis de plataformas en línea para identificar qué recursos de construcción se comparten. Clasificaron 14 tipos de recursos, con los equipos, herramientas y máquinas como los más compartidos, y encontraron que las plataformas existentes presentan vacíos de

cobertura de servicios, disponibilidad de información y soporte de idiomas. EQUISHARE atiende esos vacíos con una interfaz en español, información completa del equipo y un mercado piloto local.

Antecedente 4. Curtis y Mont (2020) analizan, en el *Journal of Cleaner Production*, los modelos de negocio de la economía colaborativa desde la sostenibilidad. Concluyen que estos modelos deben diseñarse deliberadamente para mejorar su desempeño ambiental y social, y priorizan cuatro condiciones: operar como plataforma de dos o más lados, aprovechar la capacidad ociosa de un stock de bienes existente, tener motivaciones de propiedad no solo pecuniarias y facilitar el acceso temporal sobre la propiedad. Estas condiciones vinculan la propuesta con el ODS 12 y se tradujeron en requisitos como el registro de equipos ya existentes y la reserva por días.

Antecedente 5. ter Huurne et al. (2017) realizan una revisión sistemática de 45 artículos sobre los antecedentes de la confianza en la economía colaborativa y encuentran que la confianza depende de la reputación, de la confianza en la plataforma y de la experiencia de interacción, y que no debe reducirse solo a los sistemas de reputación. Por ello, EQUISHARE combina calificaciones mutuas (RN-05), revisión automática de equipos mal calificados (RN-06), verificación del RUC (RN-01) y reglas transparentes de cobro y cancelación.

Antecedente 6. Roque Mayo y Minchán Huamán (2025) desarrollaron en la Universidad Privada del Norte un aplicativo móvil para la gestión de citas de un taller automotriz de Cajamarca, construido con Android Studio, Java y Firebase bajo la metodología Scrum. Con un diseño preexperimental de 133 clientes, la gestión de citas pasó de un 90,2 % en nivel «regular» a un 82 % en nivel «excelente», con significancia estadística según la prueba de Wilcoxon. Este antecedente local respalda el uso de Scrum y Firebase en proyectos académicos de aplicaciones móviles; EQUISHARE avanza sobre ese enfoque con Kotlin, Jetpack Compose, una API REST propia y persistencia local.

3.1.2. Casos de la industria

EquipmentShare nació como un mercado para alquilar y compartir equipos de construcción, participó en la aceleradora Y Combinator en el invierno de 2015 y evolucionó hacia una plataforma integral para obras con 373 sedes en

Estados Unidos, hasta salir a bolsa en enero de 2026 (Tan, 2026). Yard Club, otro mercado de equipos entre contratistas, procesó 120 millones de dólares en transacciones durante 2016 con 2 500 contratistas y empresas de alquiler, y fue adquirido por Caterpillar en 2017 (Lawler, 2017). En Europa, FLOOW2 opera como mercado B2B donde empresas de sectores como construcción y salud comparten equipos y personal, con herramientas de planificación, pagos en línea y verificación crediticia (Ellen MacArthur Foundation, 2021), y Sharemac reúne más de 32 000 máquinas con una comisión de 4 % a 8 % por transacción (Constructor University, 2019). Estos casos muestran que el modelo es viable a escala, aunque todos operan en mercados de alto volumen y en inglés o alemán, lo que deja un espacio para una solución local en español orientada a pymes.

3.1.3. Tendencias de las aplicaciones móviles Android

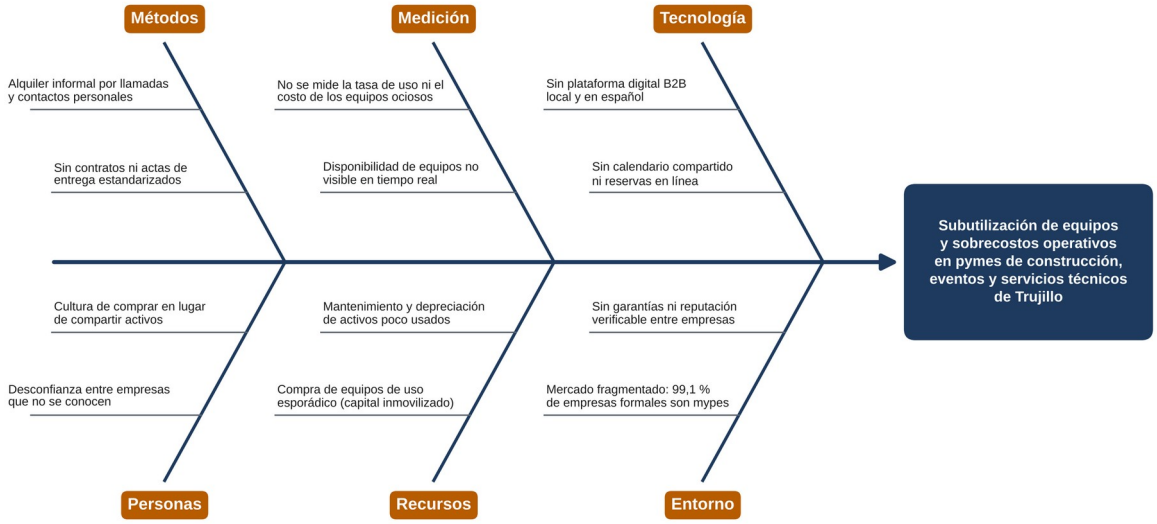
Google anunció en 2019 que el desarrollo Android sería cada vez más *Kotlin-first* y recomienda iniciar las nuevas aplicaciones en Kotlin; reporta que las aplicaciones con código Kotlin tienen 20 % menos probabilidades de fallar y que funciones como Jetpack Compose son exclusivas de Kotlin (Android Developers, s. f.-a). Compose es el kit de herramientas recomendado para construir interfaces nativas (Android Developers, s. f.-j) y, al cumplir cinco años, más del 68 % de las 1 000 aplicaciones principales lo usaban en producción (Franks et al., 2026).

En arquitectura, la guía oficial recomienda separar la capa de interfaz, con ViewModel como contenedor del estado, de la capa de datos organizada en repositorios, con flujo de datos unidireccional y una fuente única de verdad (Android Developers, s. f.-b). Las aplicaciones *offline-first* muestran primero los datos locales y usan la red solo para sincronizar (Android Developers, s. f.-c). En privacidad, el selector de fotos del sistema permite compartir solo las imágenes elegidas sin permisos amplios de almacenamiento (Android Developers, s. f.-g) y la guía de ubicación pide solicitar únicamente el nivel de precisión que la función necesita (Android Developers, s. f.-f). Finalmente, desde el 31 de agosto de 2026 Google Play exige que las nuevas aplicaciones apunten a Android 16 (API 36) (Android Developers, s. f.-i), requisito que se atenderá antes de una eventual publicación.

3.2. Identificación y formulación del problema

Para identificar las causas del problema se usó el diagrama de Ishikawa o de causa y efecto, herramienta de calidad que organiza las posibles causas de un problema en categorías (ASQ, s. f.). Las seis categorías se adaptaron al contexto de servicios: métodos, medición, tecnología, personas, recursos y entorno. La Figura 1 presenta el resultado del análisis del equipo.

Figura 1
Diagrama de Ishikawa del problema de subutilización de equipos



Nota. Elaboración propia a partir del análisis del equipo, Benoit et al. (2025), Constructor University (2019) y Agencia Andina (2025).

A partir del análisis se formula la siguiente pregunta de investigación: **¿de qué manera una aplicación móvil Android de alquiler compartido de equipos entre empresas puede contribuir a reducir la subutilización de equipos y los sobrecostos operativos de las pymes de construcción, eventos y servicios técnicos de Trujillo en 2026?** La Tabla 1 desagrega el problema general en problemas específicos.

Tabla 1
Problema general y problemas específicos

Tipo	Enunciado
Problema general	¿De qué manera una aplicación móvil Android de alquiler compartido de equipos entre empresas puede contribuir a reducir la subutilización de equipos y los sobrecostos operativos de las pymes de Trujillo en 2026?
Problema específico 1	¿Cómo publicar y encontrar equipos disponibles entre empresas sin depender de llamadas y contactos personales?

Tipo	Enunciado
Problema específico 2	¿Cómo evitar reservas duplicadas y conocer la disponibilidad real de un equipo antes de reservarlo?
Problema específico 3	¿Cómo generar confianza entre empresas que no se conocen y transparentar los costos de cada alquiler?
Problema específico 4	¿Cómo ubicar equipos cercanos a la obra o al evento para reducir tiempos y costos de traslado?
Problema específico 5	¿Cómo mantener la aplicación útil con conectividad limitada y proteger los datos de las empresas?

Nota. Elaboración propia.

3.3. Propuestas

Se evaluaron tres propuestas iniciales de solución. Todas comparten el modelo de negocio descrito; difieren en el canal y en la arquitectura técnica.

3.3.1. Propuesta 01: aplicación Android nativa con API REST propia y Firebase (seleccionada)

- **Descripción:** aplicación Android nativa (Kotlin, Jetpack Compose y MVVM) con persistencia local para trabajar sin conexión y una API REST propia en Node.js y Express; se apoya en Firebase y Google Maps.
- **Ventajas:** acceso completo a funciones del teléfono (GPS, cámara y selector de fotos); experiencia fluida y disponible sin conexión gracias a la caché local; reglas de negocio centralizadas y verificables en el servidor; cumple los requisitos académicos del proyecto (MVVM, persistencia local, API REST, Firebase y mapas) y usa servicios con planes gratuitos.
- **Desventajas:** exige mantener dos proyectos (app y API); excluye por ahora a usuarios de iOS, cerca del 14 % del mercado móvil peruano (StatCounter, 2026); el servidor propio requiere alojamiento y mantenimiento en producción.

3.3.2. Propuesta 02: aplicación web progresiva (PWA) adaptable

- **Descripción:** sitio web responsivo instalable en el teléfono, con el mismo catálogo y flujo de reservas.
- **Ventajas:** un solo código para Android, iOS y escritorio; distribución sin tienda de aplicaciones.

- **Desventajas:** menor integración con funciones nativas y con el trabajo sin conexión complejo; no desarrolla las competencias técnicas que requiere el proyecto (Kotlin, Compose, Room, Retrofit) y la experiencia depende del navegador del usuario.

3.3.3. Propuesta 03: aplicación Android solo con Firebase como backend

- **Descripción:** propuesta original del equipo, en la que la app se conecta directamente a Firestore, Storage y Cloud Functions sin servidor propio.
- **Ventajas:** menor infraestructura y sincronización en tiempo real incorporada.
- **Desventajas:** desde septiembre de 2024, Cloud Storage for Firebase exige el plan de pago Blaze incluso para los buckets predeterminados (Firebase, s. f.-b), lo que afecta a las fotos de los equipos; las reglas de negocio quedarían repartidas entre reglas de seguridad y funciones en la nube; y no contempla el consumo de una API REST con Retrofit, requisito del proyecto.

Se seleccionó la propuesta 01 porque es la única que satisface simultáneamente los requisitos académicos, mantiene el costo en cero durante el desarrollo y concentra las reglas de negocio en un servidor que puede probarse de forma automatizada. La propuesta 03 no se descartó por completo: Firebase se conserva para la autenticación y Firestore.

3.4. Definición de objetivos

3.4.1. Objetivo general

Desarrollar una aplicación móvil Android con arquitectura MVVM, integrada con una API REST propia, Firebase y Google Maps, que permita a las pymes de Trujillo publicar, buscar y reservar equipos subutilizados, con el fin de contribuir a la reducción de sobrecostos operativos y al uso eficiente de recursos promovido por el ODS 12.

3.4.2. Objetivos específicos

- **OE1.** Especificar los requisitos del producto mediante un Product Backlog de historias de usuario con criterios de aceptación, las ocho

reglas de negocio y los requisitos no funcionales vinculados con el ODS 12, la privacidad y la accesibilidad.

- **OE2.** Diseñar la arquitectura MVVM, el modelo de datos local y remoto, el contrato JSON de la API y el flujo de pantallas de la aplicación.
- **OE3.** Implementar la interfaz y la navegación en Kotlin con Jetpack Compose, con validación en el cliente de todos los formularios.
- **OE4.** Implementar la persistencia local con Room y DataStore bajo el patrón Repository para que el catálogo y las reservas estén disponibles sin conexión.
- **OE5.** Implementar la API REST en Node.js y Express con operaciones CRUD y consumirla con Retrofit, gestionando estados de carga, error, ausencia de datos y conectividad.
- **OE6.** Integrar Firebase Authentication, Cloud Firestore y Google Maps con manejo de permisos y protección de datos personales.
- **OE7.** Verificar el cumplimiento del 100 % de las reglas de negocio implementadas mediante pruebas automatizadas del servidor y de la aplicación.

IV. HERRAMIENTAS DE INGENIERÍA

4.1. Comparación y selección de metodologías

Se compararon tres marcos de trabajo ágiles aplicables a aplicaciones móviles: Scrum, Extreme Programming (XP) y Mobile-D. Scrum es un marco ligero que genera valor mediante soluciones adaptativas, organizado en sprints de un mes o menos y en tres artefactos con compromisos explícitos: el Product Backlog con el objetivo del producto, el Sprint Backlog con el objetivo del sprint y el Incremento con la definición de terminado (Schwaber y Sutherland, 2020). Mobile-D es un enfoque ágil propuesto específicamente para el desarrollo móvil por el centro VTT de Finlandia (Abrahamsson et al., 2004). La Tabla 2 resume la comparación.

Tabla 2
Comparación de metodologías de desarrollo

Criterio	Scrum	XP	Mobile-D
Enfoque	Gestión iterativa del producto por sprints	Prácticas de ingeniería (pares, TDD, integración continua)	Ciclo ágil diseñado para equipos móviles pequeños
Tamaño de equipo	Equipos pequeños con roles definidos	Equipos pequeños y colocalizados	Menos de 10 desarrolladores
Artefactos	Product Backlog, Sprint Backlog, Incremento	Historias, tarjetas CRC, pruebas	Fases de exploración, inicialización, producción, estabilización y prueba
Uso en proyectos académicos y antecedentes locales	Alto (Roque Mayo y Minchán Huamán, 2025)	Medio	Bajo
Adecuación a sprints cortos alineados con los módulos funcionales del proyecto	Alta: un sprint por módulo funcional	Media: exige programación en pares constante	Media: fases más rígidas

Nota. Elaboración propia a partir de Schwaber y Sutherland (2020) y Abrahamsson et al. (2004).

Se seleccionó **Scrum**, complementado con dos prácticas de XP (pruebas unitarias automatizadas y revisión de código en GitHub). La razón principal es que el trabajo se organiza en sprints cortos alineados con los módulos funcionales del proyecto (interfaz, persistencia, API, Firebase y mapas), cada

uno con un objetivo propio; además, el Product Backlog priorizado permite dejar como trabajo futuro funciones como el Machine Learning sin bloquear el resto del producto.

4.2. Estándares de ingeniería aplicados

Tabla 3
Estándares y guías aplicados en el proyecto

Estándar o guía	Aplicación en EQUISHARE
ISO/IEC 25010:2023, modelo de calidad del producto con nueve características (International Organization for Standardization, 2023)	Se usó para redactar requisitos no funcionales de adecuación funcional, eficiencia de desempeño, capacidad de interacción, fiabilidad, seguridad y mantenibilidad.
OWASP MASVS, ocho grupos de controles: almacenamiento, criptografía, autenticación, red, plataforma, código, resiliencia y privacidad (OWASP, s. f.)	No se guardan contraseñas ni tokens en el teléfono; HTTPS obligatorio en la versión de producción; verificación de firma del token en el servidor; permisos mínimos.
WCAG 2.2 (W3C, 2024) y guía de accesibilidad de Android (Android Developers, s. f.-h)	Contraste de texto de al menos 4,5:1, objetivos táctiles de 48 dp, descripciones de contenido en íconos y botones, y mensajes de error en texto.
Guía de arquitectura de aplicaciones de Android (Android Developers, s. f.-b)	Capas de interfaz, dominio y datos; ViewModel con UI State; repositorios como fuente única de verdad.
OpenAPI 3.0.3 (OpenAPI Initiative, 2020)	Contrato JSON de los 21 endpoints en `docs/openapi.yaml`.
Ley N.° 29733 y su reglamento, D.S. N.° 016-2024-JUS	Consentimiento libre, previo, expreso e informado; finalidad declarada; medidas de seguridad; derechos ARCO.

Nota. Elaboración propia.

4.3. Selección de herramientas de hardware y software

Tabla 4
Herramientas de software seleccionadas

Herramienta y versión	Uso	Justificación
Android Studio (versión estable vigente)	IDE, emulador, perfilador	Entorno oficial de Android, con soporte para Compose y vista previa.
Kotlin 2.0.21	Lenguaje de la app	Lenguaje recomendado por Google para Android (Android Developers, s. f.-a).
Jetpack Compose (BOM 2024.12.01) y Material 3	Interfaz declarativa	Kit recomendado para interfaces nativas (Android Developers, s. f.-j).
Android Gradle Plugin 8.7.3, compileSdk y targetSdk 35, minSdk 26	Compilación	minSdk 26 (Android 8.0) habilita java.time y los íconos adaptativos.

Herramienta y versión	Uso	Justificación
Room 2.6.1 y DataStore Preferences 1.1.1	Persistencia local	Room abstrae SQLite con verificación de consultas en compilación (Android Developers, s. f.-e); DataStore reemplaza SharedPreferences (Android Developers, s. f.-d).
Retrofit 2.11.0, OkHttp 4.12.0 y Gson	Cliente HTTP	Convierte la API HTTP en una interfaz Kotlin (Square, s. f.).
Firebase BoM 33.7.0 (Authentication y Cloud Firestore)	Identidad y datos del usuario	Plan Spark sin costo; Firestore incluye 1 GiB almacenado, 50 000 lecturas, 20 000 escrituras y 20 000 eliminaciones diarias (Firebase, s. f.-a).
Maps Compose 6.2.1 y Play Services Location 21.3.0	Mapas y GPS	Biblioteca oficial de mapas para Compose (Google for Developers, s. f.-c).
Node.js 20.19+ y Express 5	API REST	Framework minimalista para APIs en Node.js (OpenJS Foundation, s. f.).
Git y GitHub	Control de versiones	Trabajo colaborativo e historial de cambios del código.

Nota. Elaboración propia.

En hardware se utilizan las computadoras personales de los integrantes con Windows, capaces de ejecutar Android Studio y el emulador, y como dispositivos de prueba el emulador de Android con imagen de Google Play (API 35) y teléfonos Android de los integrantes con versión 8.0 o superior. El servidor de desarrollo se ejecuta en una de las computadoras dentro de la misma red Wi-Fi que los teléfonos.

4.4. Recolección, análisis y procesamiento de datos

Recolección. Los datos se recogen en formularios con validación inmediata (correo, contraseña, RUC, razón social, celular, datos del equipo y fechas), mediante el GPS solo cuando el usuario lo autoriza y mediante el selector de fotos del sistema, que entrega a la app únicamente la imagen elegida. Para validar el diagnóstico del problema se aplicará una encuesta a pymes de Trujillo.

Análisis. El servidor aplica las reglas de negocio a cada operación: verifica el dígito verificador del RUC con el algoritmo de módulo 11, detecta solapamiento de fechas, calcula la anticipación respecto a la entrega de las 08:00 en hora de Lima, aplica la comisión del 12 % y la penalidad del 20 %, recalcula el promedio de calificaciones y marca para revisión los equipos que caen por

debajo de 3,0 tras cinco reseñas. La distancia entre la empresa y cada equipo se calcula con la fórmula del haversine para ordenar el catálogo por cercanía.

Procesamiento. Los datos viajan en JSON y se transforman en tres representaciones: objetos de transferencia (DTO), entidades de Room y modelos de dominio. Las fotos se reducen en el teléfono a un máximo de 1 600 píxeles por lado y se comprimen en JPEG antes de subirlas, lo que ahorra datos móviles y respeta el límite de 5 MB del servidor.

4.5. Requisitos del usuario final y del servidor

Tabla 5
Requisitos del usuario final y del servidor

Ámbito	Requisito
Usuario final	• Teléfono Android 8.0 (API 26) o superior con Google Play Services. • Conexión a internet para publicar y reservar; consulta del catálogo guardado sin conexión. • GPS opcional: la app funciona si se niega el permiso de ubicación. • Cuenta de correo de la empresa y RUC válido para operar (RN-01).
Servidor	• Node.js 20.19 o superior; puerto 3000 accesible desde la red de los teléfonos. • Almacenamiento para el archivo de datos JSON y la carpeta de fotos. • Acceso a internet para descargar las claves públicas de Google y verificar los tokens de Firebase. • En producción: HTTPS mediante un proxy inverso o un servicio de alojamiento.
Servicios en la nube	• Proyecto de Firebase en plan Spark con Authentication por correo y Cloud Firestore con las reglas de seguridad del proyecto. • Clave de Maps SDK for Android restringida al paquete com.equishare.app; Google exige habilitar la facturación del proyecto, aunque el SKU «Maps SDK» figura con uso ilimitado sin costo (Google for Developers, s. f.-a; s. f.-b).

Nota. Elaboración propia.

V. GENERACIÓN DE SOLUCIONES

5.1. Evaluación de tecnologías de aplicaciones móviles

La Tabla 6 analiza, en una tabla de doble entrada, las ventajas y desventajas de las tecnologías evaluadas para cada capa de la solución. Las desventajas identificadas se mitigaron con decisiones de diseño.

Tabla 6
Ventajas y desventajas de las tecnologías evaluadas

Tecnología	Ventajas	Desventajas
Android Studio	• IDE oficial con emulador, inspector de base de datos y vista previa de Compose. • Asistentes de actualización de Gradle y del SDK.	• Consume bastante memoria y disco. • Las actualizaciones frecuentes pueden exigir ajustar versiones de Gradle y del JDK.
Kotlin	• Lenguaje recomendado por Google; seguridad ante valores nulos y corrutinas. • Interoperable con bibliotecas Java.	• Tiempos de compilación mayores que Java en proyectos grandes. • Requiere aprender corrutinas y Flow.
Jetpack Compose	• Interfaz declarativa que se redibuja según el estado. • Material 3, tema oscuro y accesibilidad incorporados.	• APIs de algunos componentes aún marcadas como experimentales (por ejemplo, el selector de fechas). • Exige cuidar las recomposiciones.
Flutter (alternativa evaluada)	• Un solo código para Android e iOS.	• Usa Dart, fuera del alcance del proyecto; integración nativa a través de complementos.
MVVM con ViewModel y StateFlow	• Separa interfaz, estado y datos; el estado sobrevive a la rotación. • Facilita pruebas con repositorios falsos.	• Más archivos y clases que un diseño monolítico.
Room y DataStore	• Room valida el SQL al compilar y expone Flow; DataStore es asíncrono y transaccional.	• Room requiere migraciones al cambiar el esquema; DataStore solo sirve para datos pequeños.
Retrofit y OkHttp	• Declara la API como interfaz con funciones suspend; interceptores para el token.	• Los errores HTTP llegan como excepciones que deben traducirse a mensajes.
Node.js y Express	• Rápido de desarrollar y con un gran ecosistema npm.	• Sin tipado estático; la validación de datos debe programarse.
Firebase Authentication y Cloud Firestore	• Inicio de sesión seguro sin almacenar contraseñas propias; datos con reglas de seguridad y modo sin conexión.	• Dependencia de un proveedor; cuotas diarias en el plan gratuito.
Google Maps SDK y Fused Location	• Mapas oficiales, marcadores y ubicación con bajo consumo de batería.	• Requiere clave de API y facturación habilitada en el proyecto de Google Cloud.

Nota. Elaboración propia a partir de la documentación oficial citada en la sección IV.

5.2. Propuesta de solución y restricciones realistas

La propuesta seleccionada se diseñó considerando las restricciones realistas que pide el proyecto. La Tabla 7 indica cada restricción y la decisión técnica que la atiende.

Tabla 7
Restricciones realistas y decisiones de diseño

Restricción	Decisión en EQUISHARE
Conectividad	Arquitectura offline-first: el catálogo y las reservas se leen de Room; aviso «Sin conexión»; recarga automática al volver la red; tiempos de espera de 10 s (conexión) y 20 s (lectura).
Seguridad	Firebase Authentication; verificación de firma, emisor y audiencia del token en el servidor; el servidor revalida todas las reglas; HTTPS obligatorio en producción y HTTP solo en depuración; reglas de Firestore por usuario.
Privacidad	Consentimiento sin casilla premarcada; ubicación aproximada suficiente y opcional; selector de fotos sin permiso de almacenamiento; la caché privada se borra al cerrar sesión; no se piden contactos.
Accesibilidad	Contraste ≥ 4,5:1, objetivos táctiles de 48 dp, descripciones de contenido, errores en texto y no solo en color, soporte de tema oscuro.
Rendimiento	Listas perezosas (LazyColumn), búsqueda con espera de 400 ms para no saturar la API, mapa en modo lite en el detalle, imágenes reducidas a 1 600 px.
Uso responsable de recursos	Planes gratuitos (Firebase Spark, servidor propio); compresión de fotos para ahorrar datos; ubicación con prioridad balanceada para ahorrar batería; sin seguimiento en segundo plano.
Tiempo y alcance	El plazo del curso obligó a priorizar el ciclo completo del alquiler; las funciones de mayor riesgo (Machine Learning, notificaciones push, pagos simulados y panel administrativo) quedan como trabajo futuro.

Nota. Elaboración propia.

5.3. Investigación 1: marketing irresponsable en aplicaciones y propuestas éticas

Los patrones oscuros (*dark patterns*) son decisiones de diseño que inducen a los usuarios a acciones que no tomarían con información clara. Mathur et al. (2019) analizaron cerca de 11 000 sitios de comercio electrónico y encontraron 1 818 instancias de 15 tipos de patrones oscuros en 1 254 de ellos, y la Comisión Federal de Comercio de Estados Unidos los agrupó en cuatro categorías: publicidad engañosa o disfrazada, obstáculos para cancelar suscripciones, términos y cargos ocultos, y opciones de privacidad que empujan

a compartir más datos (Federal Trade Commission, 2022). El Anexo 1 analiza cinco casos documentados.

De ese análisis surgieron seis principios que ya están implementados en la aplicación: cancelación simétrica (eliminar un equipo o cancelar una reserva toma un botón y una confirmación, igual que crearlos), disponibilidad real sin mensajes de urgencia falsos, ningún cargo o servicio preseleccionado, permisos explícitos y justificados, costos explicados antes de confirmar y transparencia en la validación automática de fotos, que siempre informa el motivo del rechazo.

VI. METODOLOGÍA DE DESARROLLO A UTILIZAR

6.1. Aplicación móvil

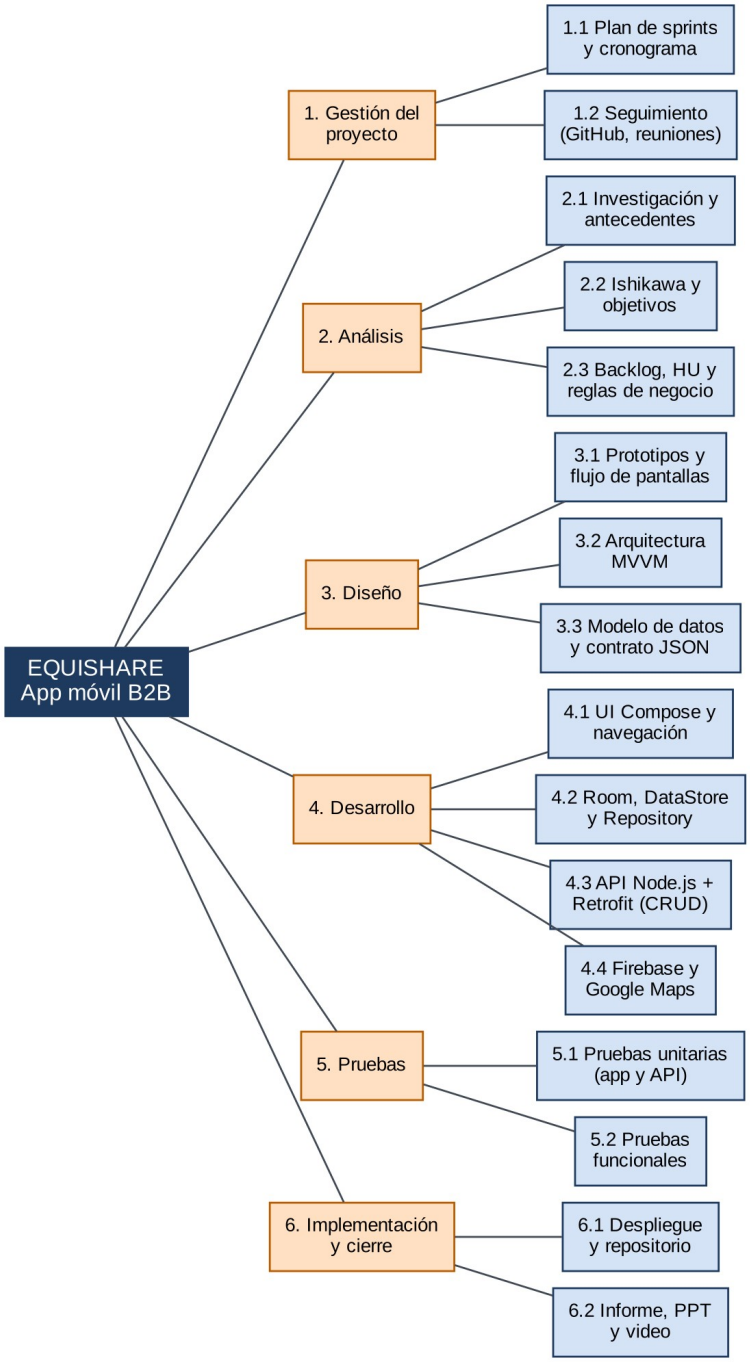
El desarrollo sigue Scrum adaptado al tamaño del equipo y al alcance del proyecto. Los integrantes asumen los roles de Product Owner, Scrum Master y equipo de desarrollo, con sprints cortos alineados con los módulos funcionales del proyecto. Cada sprint inicia con la planificación del Sprint Backlog, se sigue en reuniones breves y en el tablero del repositorio, y termina con la revisión del incremento y una retrospectiva. La definición de terminado exige que la historia cumpla sus criterios de aceptación, que el código esté en GitHub y que las pruebas automatizadas relacionadas pasen.

6.1.1. Planificación: EDT y cronograma

La Figura 2 muestra la estructura de desglose del trabajo (EDT) en seis paquetes y la Figura 3 el cronograma de sprints. La Tabla 8 detalla el objetivo y el entregable de cada sprint.

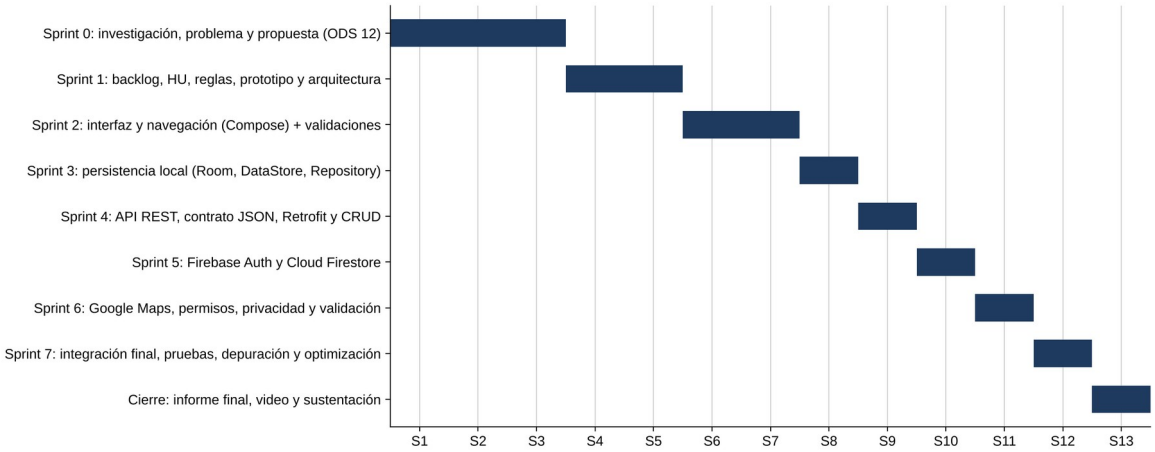
Figura 2

Estructura de desglose del trabajo (EDT) de EQUISHARE



Nota. Elaboración propia.

Figura 3
Cronograma de sprints del proyecto



Nota. Elaboración propia.

Tabla 8
Plan de sprints

Sprint	Objetivo del sprint	Entregable
0	Investigar papers, casos y tendencias; identificar oportunidades con enfoque ODS 12; definir problema, usuario y propuesta	Informe inicial, Ishikawa, propuesta
1	Product Backlog, reglas de negocio, prototipo, criterios de aceptación, stack y arquitectura	Backlog, prototipos, evidencia técnica
2	Interfaz y navegación en Compose; validaciones y componentes	Pantallas navegables con validación
3	Persistencia local con Room, DataStore y Repository; evidencia MVVM	Catálogo y reservas sin conexión
4	API REST: endpoints, contrato JSON, Retrofit; CRUD con errores, estados y conectividad	API Node.js con 21 operaciones y pruebas
5	Firebase Authentication y Cloud Firestore	Inicio de sesión, favoritos y consentimiento
6	Google Maps, permisos, privacidad y validación funcional	Mapa, «Cerca de mí», ubicación del equipo
7	Integración final (persistencia local, REST, Firebase y geolocalización), pruebas funcionales, depuración y optimización	Versión integrada y probada
Cierre	Informe final, video y sustentación	Entrega final

Nota. Elaboración propia.

6.1.2. Análisis: Product Backlog, reglas de negocio y requisitos

El Product Backlog contiene 16 historias de usuario, estimadas en puntos de historia con la escala de Fibonacci. La Tabla 9 muestra el estado de cada una; los criterios de aceptación están en el Anexo 2.

Tabla 9

Product Backlog priorizado

ID	Historia de usuario	Prioridad	Pts.	Sprint	Estado
HU-01	Como empresa, quiero registrarme y verificar mi RUC para poder publicar o alquilar equipos	Alta	5	2–5	Terminada (verificación de formato; consulta SUNAT como trabajo futuro)
HU-02	Como empresa arrendadora, quiero publicar un equipo con foto, precio y ubicación	Alta	8	2–4	Terminada
HU-03	Como sistema, quiero validar automáticamente que la foto sea apta antes de publicarla	Alta	13	4 / Futuro	Terminada (validación por reglas); validación con ML como trabajo futuro
HU-04	Como arrendataria, quiero buscar equipos por categoría, texto y ubicación	Alta	5	2–6	Terminada
HU-05	Como arrendataria, quiero ver la disponibilidad de un equipo antes de reservar	Alta	8	4	Terminada
HU-06	Como arrendataria, quiero reservar un equipo con al menos 24 h de anticipación	Alta	8	4	Terminada
HU-07	Como empresa, quiero recibir notificaciones de entrega y devolución	Media	5	Futuro	Trabajo futuro
HU-08	Como empresa, quiero calificar y comentar tras finalizar una transacción	Media	5	4	Terminada
HU-09	Como arrendataria, quiero pagar dentro de la app (pasarela simulada)	Media	8	Futuro	Trabajo futuro
HU-10	Como administrador, quiero un panel con métricas de uso	Baja	8	Futuro	Trabajo futuro
HU-11	Como sistema, quiero marcar para revisión equipos con promedio menor a 3,0 tras 5 reseñas	Baja	5	4	Terminada
HU-12	Como sistema, quiero aplicar una penalidad por cancelaciones con menos de 12 h	Baja	5	4	Terminada
HU-13	Como empresa, quiero ver los equipos cercanos en un mapa	Alta	8	6	Terminada
HU-14	Como empresa, quiero consultar el catálogo y mis reservas sin conexión	Alta	8	3	Terminada

ID	Historia de usuario	Prioridad	Pts.	Sprint	Estado
HU-15	Como empresa, quiero editar, ocultar y eliminar mis equipos	Alta	5	4	Terminada
HU-16	Como empresa, quiero guardar equipos favoritos sincronizados con mi cuenta	Media	3	5	Terminada

Nota. Elaboración propia.

La Tabla 10 indica dónde se aplica cada regla de negocio. Todas se validan en el servidor, que es la fuente de verdad, y las que afectan la experiencia también se validan en la app para dar retroalimentación inmediata sin consumir datos.

Tabla 10
Reglas de negocio e implementación

ID	Regla	Implementación
RN-01	Toda empresa debe registrarse y verificar su RUC antes de publicar o alquilar	Middleware `requiereEmpresaVerificada` (API); `Validadores.ruc` y pantalla «Registra tu empresa» (app)
RN-02	No se puede reservar un equipo con una reserva activa que se cruce con las fechas	`haySolapamiento` (API, HTTP 409); selector de fechas que deshabilita días ocupados (app)
RN-03	Comisión del 12 % del valor de la transacción a cargo de la arrendadora	`cotizar` (API y app); desglose visible antes de confirmar
RN-04	Toda foto debe pasar la validación automática antes de ser visible	`validarFoto`: formato, tamaño ≤ 5 MB, mínimo 640×480, proporción ≤ 2:1; equipo sin foto aprobada queda en «PENDIENTE_FOTO»
RN-05	Ambas empresas califican (1 a 5 + comentario) dentro de las 48 h tras la devolución	`verificarVentanaResena` (API); botón «Calificar» solo dentro del plazo (app)
RN-06	Equipo con promedio < 3,0 tras 5 transacciones pasa a revisión	`requiereRevision`: estado «EN_REVISION», oculto del catálogo
RN-07	Las reservas se hacen con al menos 24 h de anticipación	Anticipación medida respecto a la entrega de las 08:00 (hora de Lima) del primer día
RN-08	Cancelar con menos de 12 h de anticipación genera una penalidad del 20 %	`calcularPenalidad` (API); aviso previo con el monto exacto (app)

Nota. Elaboración propia.

Los requisitos no funcionales se redactaron con las características de la ISO/IEC 25010:2023 e incluyen requisitos vinculados con el ODS 12 (Tabla 11).

Tabla 11
Requisitos no funcionales

ID	Característica	Requisito y criterio de verificación
RNF-01	Fiabilidad (disponibilidad sin conexión)	El catálogo y las reservas consultadas previamente se muestran sin conexión desde Room; se verifica con el modo avión.
RNF-02	Eficiencia de desempeño	La búsqueda espera 400 ms de inactividad antes de consultar; las fotos se envían con un máximo de 1 600 px por lado.
RNF-03	Seguridad	Toda operación de escritura exige token válido; el servidor responde 401 si el token no es válido y 403 si el recurso no le pertenece.
RNF-04	Seguridad y privacidad	Sin contraseñas ni tokens en el almacenamiento local; caché privada eliminada al cerrar sesión; reglas de Firestore por usuario.
RNF-05	Capacidad de interacción (accesibilidad)	Contraste ≥ 4,5:1, objetivos táctiles ≥ 48 dp y descripción de contenido en todos los íconos interactivos.
RNF-06	Compatibilidad	Android 8.0 (API 26) o superior; API REST con contrato OpenAPI 3.0.
RNF-07	Mantenibilidad	Arquitectura por capas con interfaces de repositorio; pruebas automatizadas de reglas de negocio.
RNF-08	ODS 12 (uso eficiente de recursos)	Las empresas registran equipos existentes y los ofrecen por días; la app muestra la reutilización como acceso temporal y no como compra.
RNF-09	ODS 12 (información para el consumo responsable)	El costo total, la comisión y las penalidades se muestran antes de confirmar; no se usan mensajes de urgencia ni opciones premarcadas.

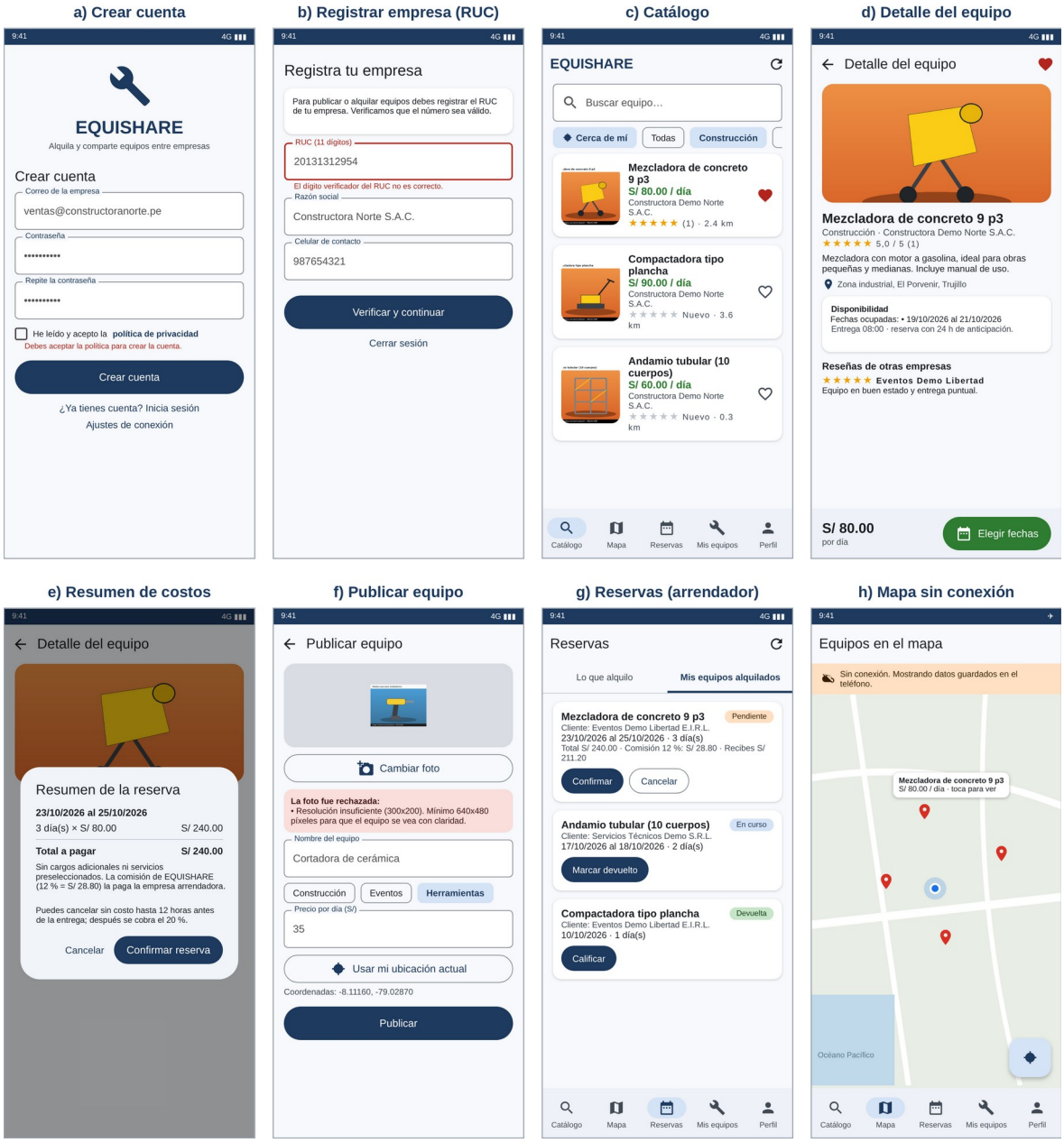
Nota. Elaboración propia.

6.1.3. Diseño: prototipos, flujo de pantallas y arquitectura

Los prototipos de baja fidelidad se convirtieron en prototipos de alta fidelidad con la paleta de la marca, contrastes accesibles y los estados de carga, error, vacío y sin conexión. La Figura 4 presenta ocho pantallas clave: creación de cuenta con consentimiento sin premarcar, registro de la empresa con validación del RUC, catálogo con filtros y favoritos, detalle con disponibilidad real, resumen transparente de costos, publicación con rechazo explicado de la foto, gestión de reservas del arrendador y mapa con aviso de modo sin conexión.

Figura 4

Prototipos de alta fidelidad de las pantallas principales

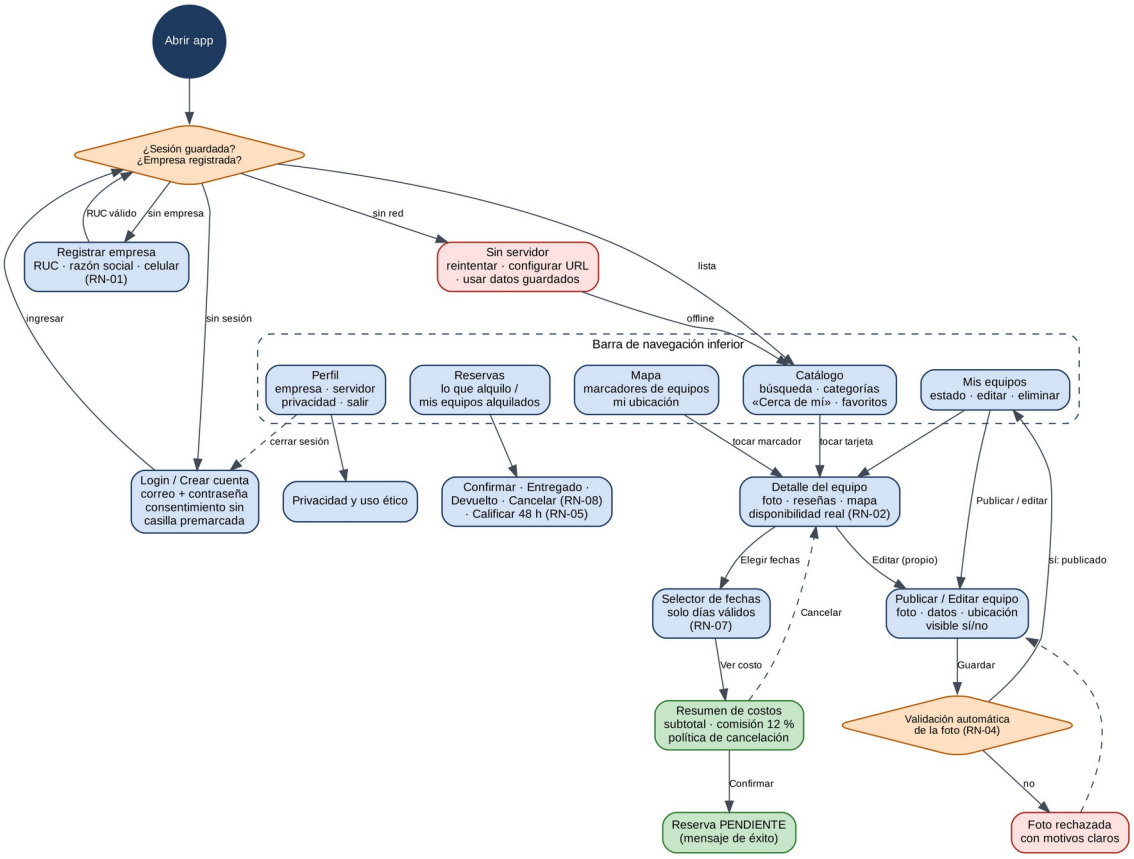


Nota. Elaboración propia. Los datos que se muestran son de demostración.

El flujo de navegación completo, con las decisiones, los caminos alternativos y las reglas de negocio que actúan en cada paso, se muestra en la Figura 5. La aplicación decide al iniciar si debe mostrar el inicio de sesión, el registro de la empresa o la aplicación principal; esta última se organiza en cinco pestañas inferiores.

Figura 5

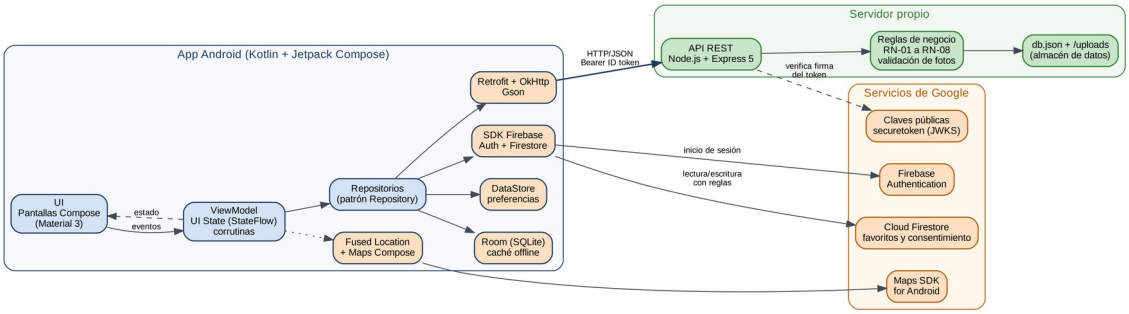
Flujo de pantallas y navegación de la aplicación



Nota. Elaboración propia.

La Figura 6 presenta la arquitectura general. La aplicación se comunica con la API REST mediante HTTP y JSON enviando el token de Firebase en la cabecera Authorization; la API verifica la firma del token con las claves públicas de Google, aplica las reglas de negocio y persiste los datos. Firebase Authentication y Cloud Firestore se consumen directamente desde la app con sus SDK, y el mapa se dibuja con el Maps SDK.

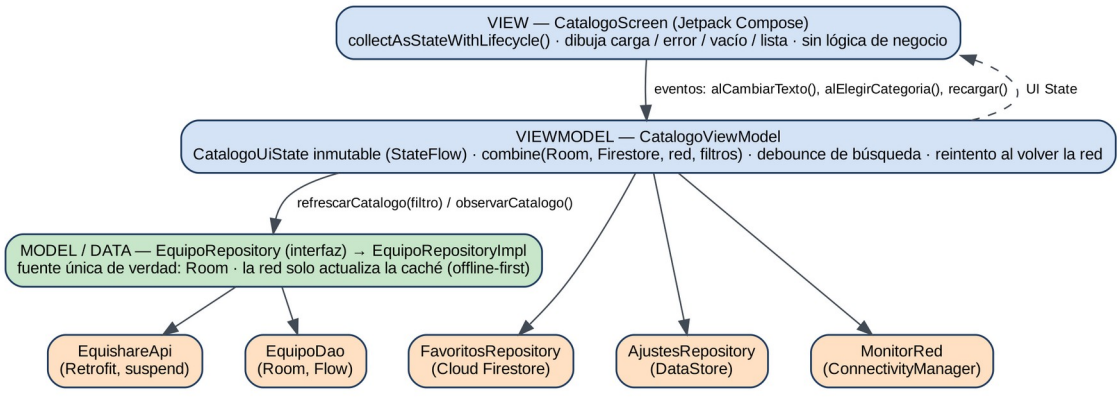
Figura 6
Arquitectura general de la solución



Nota. Elaboración propia.

La Figura 7 detalla el patrón MVVM con el caso del catálogo. La pantalla solo dibuja un estado inmutable y envía eventos; el ViewModel combina en un único `CatalogoUiState` los datos de Room, los favoritos de Firestore, el estado de la red y los filtros; y el repositorio decide cuándo consultar la API y siempre entrega los datos desde Room, siguiendo la recomendación de una fuente única de verdad (Android Developers, s. f.-b).

Figura 7
Patrón MVVM aplicado a la pantalla de catálogo

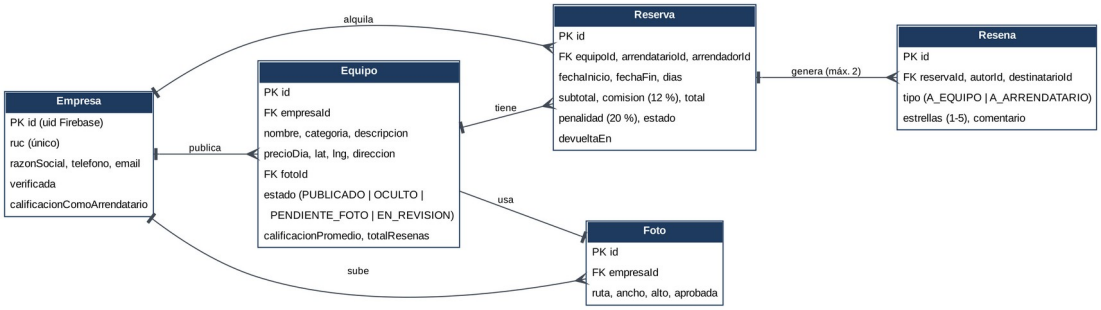


Nota. Elaboración propia.

El modelo de datos del servidor tiene cinco entidades (Figura 8). En el teléfono, Room guarda dos tablas de caché con clave compuesta para distinguir el catálogo público de los equipos propios y las reservas como arrendatario o arrendador; DataStore guarda la dirección del servidor y el último filtro, y Firestore guarda los favoritos y el consentimiento de cada usuario (Figura 9).

Figura 8

Modelo de datos del servidor



Nota. Elaboración propia.

Figura 9

Modelo de datos local (Room y DataStore) y documentos de Cloud Firestore



Nota. Elaboración propia.

El contrato JSON de la API se documentó en OpenAPI 3.0.3. La Tabla 12 resume los endpoints y los códigos de estado que devuelve cada uno; todos los

errores usan el mismo formato `{"error": {"codigo", "mensaje", "detalles"}}`.

Tabla 12
Endpoints de la API REST

Método y ruta	Descripción	Respuestas
GET /api/health	Estado del servicio y modo de autenticación	200
GET · PUT /api/empresas/me	Consultar o registrar la empresa y verificar el RUC (RN-01)	200, 201, 400, 401, 404, 409
GET /api/empresas/{id}	Perfil público de una empresa	200, 404
GET /api/equipos	Catálogo con filtros q, categoria, lat, lng y radioKm	200, 400
GET /api/equipos/mios	Equipos de la empresa autenticada	200, 401
POST /api/equipos/fotos	Subir foto y validarla (RN-04)	201, 401, 413, 422
POST /api/equipos	Crear equipo (Create)	201, 400, 401, 422
GET /api/equipos/{id}	Detalle del equipo (Read)	200, 404
PUT /api/equipos/{id}	Actualizar equipo propio (Update)	200, 400, 403, 404
DELETE /api/equipos/{id}	Eliminar equipo propio (Delete)	204, 403, 404, 409
GET /api/equipos/{id}/disponibilidad	Fechas ocupadas (RN-02)	200, 404
GET /api/equipos/{id}/resenas	Reseñas del equipo	200, 404
POST /api/reservas/cotizacion	Cálculo de días, subtotal y comisión sin reservar	200, 404, 422
POST /api/reservas	Crear reserva (RN-02, RN-03, RN-07)	201, 400, 404, 409, 422
GET /api/reservas/mias?rol=	Reservas como arrendataria o arrendadora	200, 400
GET /api/reservas/{id}	Detalle de una reserva (solo participantes)	200, 403, 404
PATCH /api/reservas/{id}/cancelar	Cancelar con penalidad si corresponde (RN-08)	200, 403, 422
PATCH /api/reservas/{id}/estado	Confirmar, entregar o registrar la devolución	200, 403, 422
POST /api/reservas/{id}/resenas	Calificar (RN-05, RN-06)	201, 400, 409, 422

Nota. Elaboración propia. Detalle completo en `equishare-api/docs/openapi.yaml`; incluye además GET /api/categorias.

6.1.4. Desarrollo: resultados por sprint

Sprint 2: interfaz, navegación y validaciones. Se construyeron 11 pantallas en Jetpack Compose con Material 3 y Navigation Compose: raíz, inicio

de sesión y creación de cuenta, registro de empresa, catálogo, mapa, reservas, mis equipos, perfil, detalle, publicación y edición, y privacidad, además de los diálogos de servidor, fechas, cotización y calificación. Las validaciones están centralizadas en el objeto `Validadores` de la capa de dominio (correo, contraseña, RUC con dígito verificador, celular, nombre, descripción, precio con coma o punto decimal, comentario y URL del servidor) y cada campo muestra su mensaje de error debajo, sin depender solo del color. Los componentes reutilizables (`EstadoCargando`, `EstadoError`, `EstadoVacio`, `BannerSinConexion`, `TarjetaEquipo`, `Estrellas`) garantizan la misma experiencia en todas las pantallas.

Sprint 3: persistencia local. `EquishareDatabase` define dos entidades y dos DAO con consultas que devuelven Flow, y las operaciones de reemplazo se ejecutan dentro de una transacción. DataStore Preferences guarda la URL del servidor, el último filtro de categoría y la sesión del modo demostración. Los repositorios implementan interfaces definidas en la capa de dominio, de modo que los ViewModel no conocen Room ni Retrofit:

```
class EquipoRepositoryImpl(api: ApiProvider, dao: EquipoDao, ...) :
EquipoRepository {
    // La UI siempre lee de Room (fuente única de verdad)
    override fun observarCatalogo(): Flow<List<Equipo>> =
        dao.observar(LISTA_CATALOGO).map { lista -> lista.map
{ it.aDominio() } }

    // La red solo actualiza la caché
    override suspend fun refrescarCatalogo(filtro: FiltroCatalogo):
Resultado<Unit> =
        when (val r = llamarApi { api.api().listarEquipos(filtro.texto,
filtro.categoria?.name, ...) }) {
            is Resultado.Exito -> { dao.reemplazarLista(LISTA_CATALOGO,
r.datos.items.map { ... }); Resultado.Exito(Unit) }
            is Resultado.Error -> r
        }
}
```

Sprint 4: API REST, CRUD, errores, estados y conectividad. La API se implementó en Node.js con Express 5 en 14 módulos: dominio (reglas, RUC y validaciones), servicios (validación de fotos), middleware de autenticación, rutas y un almacén de datos en archivo JSON con escritura atómica que podrá reemplazarse por una base de datos gestionada sin cambiar las rutas. En la app, `EquishareApi` declara los endpoints como funciones `suspend` de Retrofit y la función `llamarApi` convierte cada falla en un `Resultado.Error` con un mensaje comprensible: los códigos 400, 401, 403, 404, 409, 422 y 5xx se traducen a tipos

de error, y la falta de conexión y el tiempo agotado se distinguen de los errores del servidor. Cada pantalla resuelve cuatro estados: carga inicial, error sin datos con botón de reintento, lista vacía con sugerencia y datos con indicador de actualización; si la red falla y existen datos guardados, se muestran con un aviso.

```
suspend fun <T> llamarApi(bloque: suspend () -> T): Resultado<T> = try {
    Resultado.Exito(bloque())
} catch (e: HttpException) { Resultado.Error(errorDesdeHttp(e))      // 4xx /
5xx con mensaje de la API
} catch (e: SocketTimeoutException)
{ Resultado.Error(ErrorApp(TipoError.TIEMPO_AGOTADO))
} catch (e: IOException) { Resultado.Error(ErrorApp(TipoError.SIN_CONEXION)) }
```

Sprint 5: Firebase. `FirebaseAuthRepository` inicia sesión y crea cuentas con correo y contraseña, traduce las excepciones de Firebase a mensajes en español y entrega el ID token a OkHttp mediante un interceptor. En el servidor, el middleware verifica la firma RS256 del token con las claves públicas de Google y valida el emisor, la audiencia y la vigencia, como permite la documentación de Firebase para bibliotecas JWT de terceros (Firebase, s. f.-c). Cloud Firestore guarda los favoritos y el registro del consentimiento en `usuarios/{uid}`, protegidos por reglas que solo permiten a cada usuario leer y escribir sus propios documentos (Firebase, s. f.-d). Si no encuentra el archivo `google-services.json`, la app funciona en un modo de demostración con empresas de ejemplo, útil para probarla sin credenciales.

Sprint 6: geolocalización, permisos y privacidad. La pestaña Mapa usa Maps Compose para dibujar los equipos del catálogo con marcadores que abren el detalle, y el detalle muestra un mapa en modo lite. La función «Cerca de mí» y el botón «Usar mi ubicación actual» solicitan los permisos de ubicación en tiempo de ejecución solo cuando el usuario los usa; si se niegan, la app sigue funcionando y lo explica. La ubicación se obtiene con prioridad balanceada, no se guarda en el servidor y solo se envía como parámetro de búsqueda.

Sprint 7: integración final. Se configuró el proyecto de Firebase en el plan Spark con Authentication por correo y contraseña y Cloud Firestore con sus reglas de seguridad publicadas; desde entonces el servidor verifica cada token con el identificador del proyecto. Se habilitó Maps SDK for Android con su clave de API y la app, compilada con ambos servicios activos, se ejecutó en el emulador Pixel 8 (API 35): la pestaña Mapa muestra los equipos con marcadores y la primera cuenta real quedó registrada en Firebase Authentication con el perfil

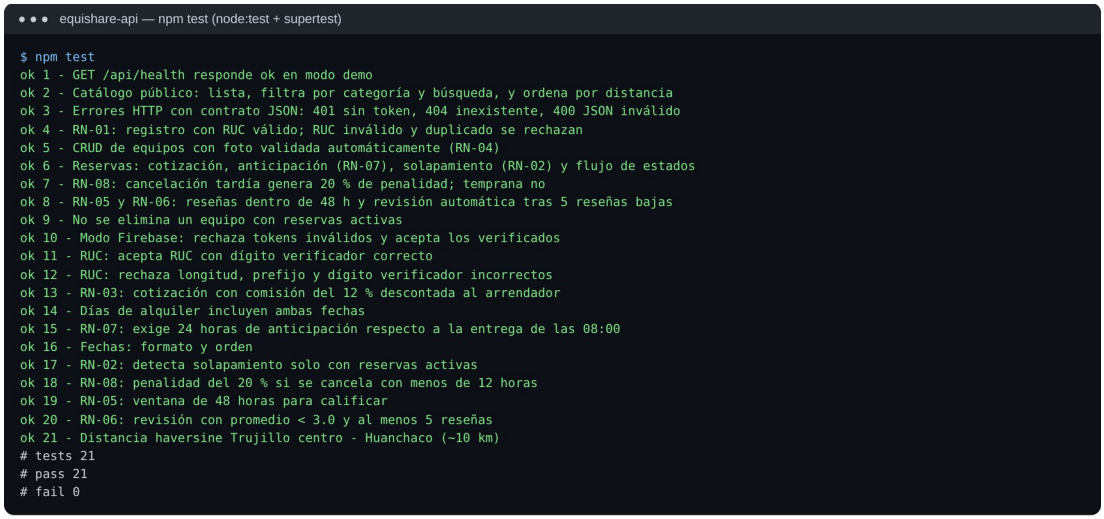
de su empresa en el servidor. Con esta integración, la aplicación opera de extremo a extremo con cuentas reales.

6.1.5. Pruebas

Se ejecutaron tres niveles de prueba. En el servidor, 21 pruebas automatizadas con `node:test` y `supertest` verifican las reglas de negocio, el CRUD, los códigos de error y la verificación de tokens en modo Firebase; las 21 pasaron (Figura 10). En la app, 16 pruebas unitarias de la capa de dominio verifican las reglas, los validadores, la traducción de códigos HTTP y el formato de montos y fechas; se ejecutaron con el compilador Kotlin 2.0.21 y las 16 pasaron (Figura 11). Otras 7 pruebas verifican los ViewModel de catálogo e inicio de sesión con repositorios falsos y `kotlinx-coroutines-test`; se ejecutan en Android Studio con `./gradlew testDebugUnitTest`.

Figura 10

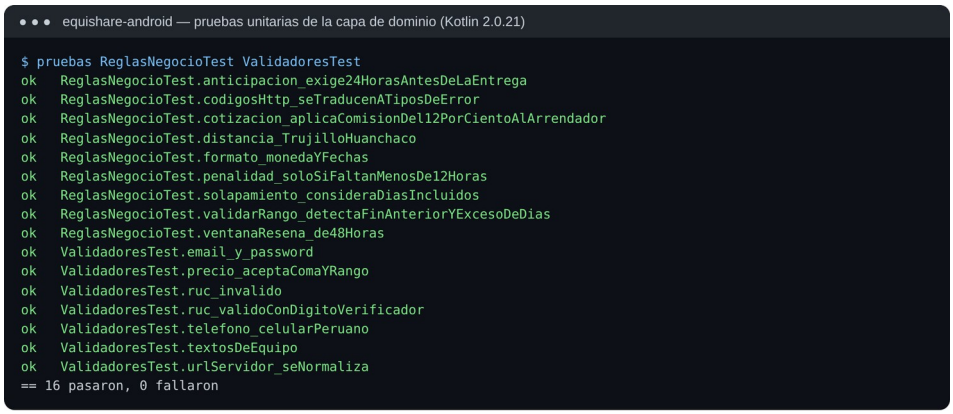
Resultado de las pruebas automatizadas de la API REST



Nota. Captura de la ejecución de `npm test` en el servidor del proyecto.

Figura 11

Resultado de las pruebas unitarias de la capa de dominio de la app



Nota. Captura de la ejecución de las clases ReglasNegocioTest y ValidadoresTest.

La Tabla 13 presenta los casos de prueba funcionales de extremo a extremo y el medio con que se verifica cada uno. Los casos de la API se ejecutaron con el servidor en funcionamiento (Figura 12) y los de la app se comprueban en el emulador, como muestran las evidencias del apartado 6.2.

Tabla 13

Casos de prueba funcionales

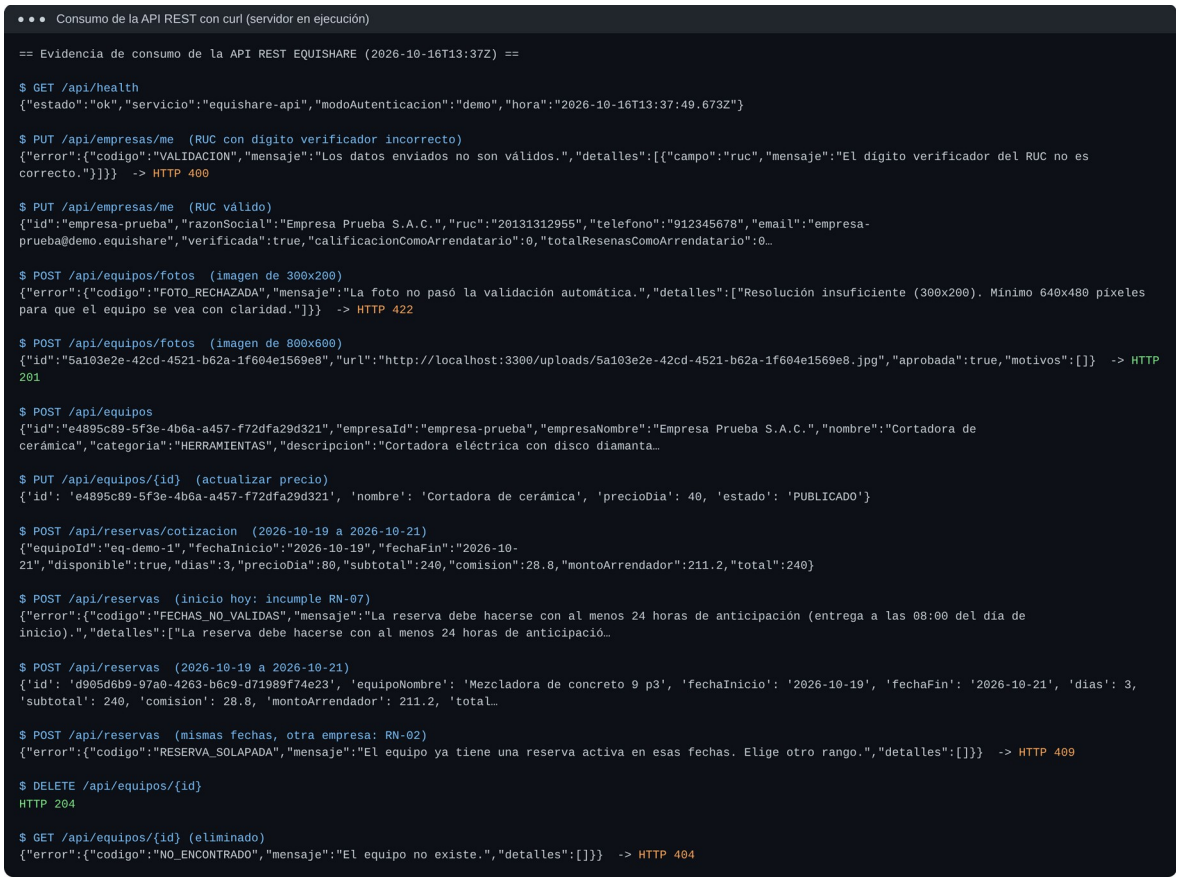
ID	Caso	Resultado esperado	Verificación
CP-01	Registrar empresa con RUC 20131312954 (dígito incorrecto)	Mensaje «El dígito verificador del RUC no es correcto»; HTTP 400	En la API
CP-02	Subir foto de 300×200 píxeles	Rechazo con motivo de resolución; HTTP 422	En la API
CP-03	Publicar equipo con foto válida	Equipo PUBLICADO y visible en el catálogo; HTTP 201	En la API
CP-04	Reservar para el mismo día	Mensaje de 24 h de anticipación (RN-07); HTTP 422	En la API
CP-05	Reservar fechas ya ocupadas	Mensaje de reserva solapada (RN-02); HTTP 409	En la API
CP-06	Cotizar 3 días de un equipo de S/ 80	Total S/ 240,00; comisión S/ 28,80; arrendador recibe S/ 211,20	En la API
CP-07	Cancelar 8 h antes de la entrega	Penalidad del 20 % (RN-08)	Prueba automatizada
CP-08	Eliminar equipo y consultarlo	HTTP 204 y luego 404	En la API
CP-09	Abrir el catálogo en modo avión	Aviso «Sin conexión» y datos guardados en Room	En la app (emulador)
CP-10	Negar el permiso de ubicación en «Cerca de mí»	La app no se cierra y sigue mostrando el catálogo completo	En la app (emulador)
CP-11	Conceder ubicación y activar «Cerca de mí»	Equipos ordenados por distancia dentro de 15 km	En la API

ID	Caso	Resultado esperado	Verificación
CP-12	Abrir la pestaña Mapa	Marcadores de los equipos; al tocar la ventana de información se abre el detalle	En la app (emulador)

Nota. Elaboración propia.

Figura 12

Consumo de la API REST con el servidor en funcionamiento



Nota. Captura de las peticiones ejecutadas el 16 de octubre de 2026 contra el servidor local del proyecto.

6.1.6. Implementación

El repositorio incluye la guía `GUIA_CONFIGURACION.md` con los pasos para ejecutar el servidor (`npm install`, `npm test`, `npm start`), crear el proyecto de Firebase, generar la clave de Google Maps restringida al paquete de la app y ejecutar la aplicación en el emulador (`http://10.0.2.2:3000/`) o en un teléfono dentro de la misma red. La dirección del servidor se puede cambiar desde la app sin recompilar, en «Ajustes de conexión». La versión compilada para producción solo permite tráfico HTTPS gracias a la configuración de seguridad de red, mientras que la versión de depuración admite HTTP para las pruebas en la red local.

6.1.7. Protección de datos personales

La Ley N.° 29733 y su reglamento aprobado por el D.S. N.° 016-2024-JUS exigen que el consentimiento sea libre, previo, expreso, inequívoco e informado, y que el responsable adopte medidas de seguridad sobre los datos (Congreso de la República del Perú, 2011; Ministerio de Justicia y Derechos Humanos, 2024). En el producto esto se evidencia en seis medidas: la casilla de aceptación de la política de privacidad no viene marcada y el registro de su aceptación se guarda con fecha y versión en Firestore; la política explica qué datos se usan, para qué, dónde se guardan y cómo ejercer los derechos ARCO; se solicita el mínimo de datos (correo, RUC, razón social y celular); la ubicación es opcional y no se almacena; las reservas y los equipos propios guardados en el teléfono se borran al cerrar sesión y la copia de seguridad automática está desactivada; y cada empresa solo accede a las reservas en las que participa (HTTP 403 en otro caso). En la investigación, los datos de demostración son ficticios y la encuesta a pymes se aplicará con consentimiento informado y sin recopilar datos personales innecesarios.

6.2. Evidencias del funcionamiento

Las evidencias del servidor (Figuras 10 y 12) se generaron con el código del repositorio. Las capturas siguientes muestran la aplicación en funcionamiento en el emulador, siguiendo el recorrido de la demostración del producto.

Evidencia 1. Búsqueda, filtros por categoría, favoritos y calificaciones (HU-04, HU-16).



Evidencia 2. Disponibilidad real, desglose transparente con comisión del 12 % y política de cancelación (HU-05, HU-06, RN-02, RN-03).



Evidencia 3. Reserva con menos de 24 horas de anticipación rechazada en la app y en el servidor (RN-07).

Evidencia 4. Flujo Confirmar → Marcar entregado → Marcar devuelto y calificación (HU-08, RN-05).

Evidencia 5. Rechazo con motivo explicado y publicación exitosa (HU-02, HU-03, RN-04).

Evidencia 6. Aviso «Sin conexión» y datos leídos desde Room (HU-14).

Evidencia 7. Geolocalización con Google Maps y permisos en tiempo de ejecución (HU-13).

6.3. Repositorios

El control de versiones se realiza con Git y el repositorio remoto se aloja en GitHub con dos carpetas: `equishare-android` (aplicación) y `equishare-api` (servidor). El archivo `.gitignore` excluye las credenciales (`google-services.json`, `.env` y `local.properties`). El enlace del repositorio figura en el apartado 7.4.

VII. COMPLEMENTO

7.1. Evidencia de participación

7.2. Video de presentación del proyecto (link)

El video de presentación dura como máximo 10 minutos (3 minutos de parte teórica y 7 minutos de demostración del producto) y está alojado en Office 365. Enlace: **[enlace del video]**.

7.3. PPT de sustentación

La presentación de la sustentación tiene 24 diapositivas para una exposición de 30 minutos. La Figura 13 muestra una vista general.

Figura 13
Vista general de las diapositivas de sustentación



Nota. Elaboración propia. Archivo `EQUISHARE_Sustentacion.pptx`.

7.4. Link del repositorio donde se encuentran los proyectos (código)

Repositorio GitHub: **[enlace del repositorio]**. Carpetas `equishare-android` (aplicación Kotlin) y `equishare-api` (API Node.js).

VIII. RESULTADOS

8.1. Resultados en base a los objetivos del proyecto

Tabla 14
Resultados por objetivo específico

Objetivo	Resultado
OE1. Requisitos	Product Backlog con 16 historias (12 terminadas, 1 parcial en la validación de fotos y 3 como trabajo futuro), 8 reglas de negocio, 9 requisitos no funcionales (2 vinculados con el ODS 12) y criterios de aceptación en formato Dado-Cuando-Entonces para 12 historias (Anexo 2).
OE2. Diseño	Arquitectura MVVM en tres capas (44 archivos Kotlin, unas 5 600 líneas), modelo de datos con 5 entidades en el servidor, 2 tablas Room y 1 almacén DataStore, contrato OpenAPI con 21 operaciones y flujo de 11 pantallas y 4 diálogos.
OE3. Interfaz y validaciones	11 pantallas en Jetpack Compose con navegación inferior de 5 pestañas; 13 funciones de validación reutilizadas por todos los formularios; componentes comunes para carga, error, vacío y sin conexión.
OE4. Persistencia local	Catálogo, equipos propios y reservas en Room con lectura desde Flow; preferencias en DataStore; la UI lee siempre de la base local (offline-first).
OE5. API REST y CRUD	API Node.js + Express con CRUD completo de equipos (crear, leer, actualizar y eliminar) y de reservas (crear, leer, actualizar estado y cancelar), 8 códigos de error HTTP manejados y 21/21 pruebas aprobadas; consumo con Retrofit y corrutinas.
OE6. Firebase y Google Maps	Inicio de sesión y registro con Firebase Authentication, verificación criptográfica del token en el servidor, favoritos y consentimiento en Firestore con reglas por usuario; mapa con marcadores, «Cerca de mí» en un radio de 15 km y selección de la ubicación del equipo.
OE7. Verificación	Las 8 reglas de negocio tienen pruebas automatizadas en el servidor (100 %); 16/16 pruebas de dominio de la app aprobadas, y 7 pruebas de ViewModel con repositorios simulados que se ejecutan en Android Studio.

Nota. Elaboración propia.

8.2. Discusión de resultados

La postura del proyecto es que una plataforma digital B2B puede convertir la capacidad ociosa de las pymes en un servicio de acceso temporal, siempre que resuelva la confianza entre empresas y la transparencia de la información. Los resultados son coherentes con el marco de Benoit et al. (2025), que identifica a la plataforma, a las empresas que comparten y a las que pagan por el acceso como actores indispensables: EQUISHARE permite que una misma empresa actúe en ambos roles y separa en la interfaz las reservas como arrendataria y como arrendadora.

Respecto a Goswami y Arputham (2026), que proponen la disponibilidad, el precio y la calidad y simetría de la información como palancas de adopción, el producto implementa la disponibilidad real por fechas, la cotización previa y el mismo desglose de costos para ambas partes. Sin embargo, su marco incluye servicios agrupados como operadores, mantenimiento y seguros, que EQUISHARE aún no ofrece y que podrían ser decisivos para maquinaria pesada; por ello el piloto se orienta a equipos medianos y herramientas.

Frente a los vacíos que Rathnayake et al. (2024) encontraron en las plataformas de compartición para construcción (cobertura, información e idioma), la solución ofrece información completa del equipo, reseñas y un servicio en español, aunque su cobertura se limita por ahora a Trujillo. En cuanto a la confianza, ter Huurne et al. (2017) advierten que no depende solo de la reputación; el producto combina calificaciones mutuas con la verificación del RUC y reglas explícitas de cobro y cancelación, pero la verificación aún es estructural y no consulta el padrón de SUNAT, lo que limita la garantía de identidad.

Desde la sostenibilidad, el diseño cumple tres de las cuatro condiciones de Curtis y Mont (2020): plataforma de dos lados, aprovechamiento de bienes existentes y acceso temporal. La cuarta, motivaciones de propiedad no solo pecuniarias, depende del comportamiento de las empresas y no puede asegurarse con software. Tampoco es posible afirmar todavía una reducción medible de sobrecostos o de compras de equipos, porque el proyecto se limitó a la validación técnica; la prueba con pymes reales, prevista como trabajo futuro, deberá medir indicadores como la tasa de uso de los equipos publicados y el ahorro frente a la compra.

En lo metodológico, el resultado coincide con el antecedente de Roque Mayo y Minchán Huamán (2025) en que Scrum y Firebase son viables para un equipo académico. La diferencia está en la arquitectura: separar las reglas de negocio en una API propia permitió probarlas de forma automatizada y evitó el requisito del plan de pago de Cloud Storage (Firebase, s. f.-b), a cambio de mantener un servidor adicional.

IX. CONCLUSIONES

- **Respecto al OE1**, el análisis con el diagrama de Ishikawa y los antecedentes permitió traducir el problema de subutilización en 16 historias de usuario, 8 reglas de negocio y 9 requisitos no funcionales; los requisitos vinculados con el ODS 12 se expresan como acceso temporal a bienes existentes y como información completa de costos antes de reservar.
- **Respecto al OE2**, la arquitectura MVVM con repositorios definidos como interfaces separa la interfaz de las fuentes de datos, y el contrato OpenAPI de 21 operaciones permitió desarrollar la app y la API en paralelo con un mismo formato de errores.
- **Respecto al OE3**, las 11 pantallas en Jetpack Compose cubren el ciclo completo del alquiler, y la centralización de las validaciones en la capa de dominio hizo que el cliente y el servidor apliquen los mismos criterios.
- **Respecto al OE4**, la estrategia offline-first con Room y DataStore mantiene la aplicación útil sin conexión, condición relevante para el trabajo en obras con señal limitada.
- **Respecto al OE5**, la API implementa el CRUD completo de equipos y el ciclo de vida de las reservas con manejo explícito de los códigos 400, 401, 403, 404, 409, 413, 422 y 500, y la app traduce cada uno a un estado comprensible para el usuario.
- **Respecto al OE6**, Firebase Authentication con verificación del token en el servidor y Firestore con reglas por usuario completan la integración con Firebase, con medidas de privacidad alineadas con la Ley N.° 29733; la integración con Google Maps muestra los equipos en el mapa y ordena el catálogo por cercanía, con permisos solicitados en contexto.
- **Respecto al OE7**, el 100 % de las reglas de negocio está cubierto por pruebas automatizadas del servidor (21/21 aprobadas) y la capa de dominio de la app superó 16 de 16 pruebas.

Recomendaciones

- Incorporar las pruebas de la app y del servidor a un flujo de integración continua en GitHub para ejecutarlas en cada cambio del código.
- Restringir la clave de Google Maps al paquete com.equishare.app y a la huella SHA-1 de la app, y configurar un presupuesto con alerta en Google Cloud.
- Aplicar la encuesta a pymes de Trujillo con consentimiento informado para validar el diagnóstico y medir la línea base de uso de equipos.
- Actualizar el targetSdk a API 36 y migrar el almacén de datos del servidor a una base de datos gestionada con HTTPS antes de cualquier despliegue público.
- Abordar como trabajo futuro la validación de fotos con TensorFlow Lite, las notificaciones push, la pasarela de pagos simulada, el panel administrativo y la prueba de usabilidad con pymes, manteniendo la explicación del motivo de rechazo definida en los principios éticos.

X. REFERENCIAS O BIBLIOGRAFÍA

Abrahamsson, P., Hanhineva, A., Hulkko, H., Ihme, T., Jäälinoja, J., Korkala, M., Koskela, J., Kyllönen, P., & Salo, O. (2004). Mobile-D: An agile approach for mobile application development. En J. Vlissides (Ed.), *Companion to the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications* (pp. 174–175). Association for Computing Machinery. https://doi.org/10.1145/1028664.1028736

Agencia Andina. (2025, 17 de mayo). *Perú: número de empresas formales creció el 2024 y suman 2.34 millones*. https://andina.pe/agencia/noticia-peru-numero-empresas-formales-crecio-2024-y-suman-234-millones-1029970.aspx

Android Developers. (s. f.-a). *Android's Kotlin-first approach*. https://developer.android.com/kotlin/first

Android Developers. (s. f.-b). *Guide to app architecture*. https://developer.android.com/topic/architecture

Android Developers. (s. f.-c). *Build an offline-first app*. https://developer.android.com/topic/architecture/data-layer/offline-first

Android Developers. (s. f.-d). *DataStore*. https://developer.android.com/topic/libraries/architecture/datastore

Android Developers. (s. f.-e). *Save data in a local database using Room*. https://developer.android.com/training/data-storage/room

Android Developers. (s. f.-f). *Request location permissions*. https://developer.android.com/develop/sensors-and-location/location/permissions

Android Developers. (s. f.-g). *Photo picker*. https://developer.android.com/training/data-storage/shared/photo-picker

Android Developers. (s. f.-h). *Make apps more accessible*. https://developer.android.com/guide/topics/ui/accessibility/apps

Android Developers. (s. f.-i). *Meet Google Play's target API level requirement*. https://developer.android.com/google/play/requirements/target-sdk

Android Developers. (s. f.-j). *Jetpack Compose*. https://developer.android.com/compose

ASQ. (s. f.). *What is a fishbone diagram? Ishikawa cause & effect diagram*. https://asq.org/quality-resources/fishbone

Benoit, S., Merfeld, K., Tunn, V. S. C., Schaefers, T., & Andreassen, T. W. (2025). The B2B sharing economy: Framework, implications, and future research. *Journal of Business Research, 191*, Artículo 115244. https://doi.org/10.1016/j.jbusres.2025.115244

Competition and Markets Authority. (2019, 6 de febrero). *Hotel booking sites to make major changes after CMA probe*. GOV.UK. https://www.gov.uk/government/news/hotel-booking-sites-to-make-major-changes-after-cma-probe

Congreso de la República del Perú. (2011). *Ley N.° 29733, Ley de Protección de Datos Personales*. https://www.congreso.gob.pe/Docs/DGP/DIDP/files/ley_29733.pdf

Constructor University. (2019, 21 de enero). *Sharemac: My excavator is your excavator*. https://constructor.university/news/sharemac-my-excavator-your-excavator

Curtis, S. K., & Mont, O. (2020). Sharing economy business models for sustainability. *Journal of Cleaner Production, 266*, Artículo 121519. https://doi.org/10.1016/j.jclepro.2020.121519

Ellen MacArthur Foundation. (2021, 5 de octubre). *Business-to-business asset sharing: FLOOW2*. https://ellenmacarthurfoundation.org/case-studies/business-to-business-asset-sharing

European Commission. (2022, 1 de julio). *Consumer protection: Amazon Prime changes its cancellation practices to comply with EU consumer rules* [Comunicado de prensa IP/22/4186]. https://ec.europa.eu/commission/presscorner/api/files/document/print/en/ip_22_4186/IP_22_4186_EN.pdf

European Parliamentary Research Service. (2012, 5 de septiembre). *Travel agents may not automatically include insurance*. https://epthinktank.eu/2012/09/05/travel-agents-may-not-automatically-include-insurance/

Federal Trade Commission. (2022, 15 de septiembre). *FTC report shows rise in sophisticated dark patterns designed to trick and trap consumers* [Comunicado de prensa]. https://www.ftc.gov/news-events/news/press-releases/2022/09/ftc-report-shows-rise-sophisticated-dark-patterns-designed-trick-trap-consumers

Firebase. (s. f.-a). *Firebase pricing*. https://firebase.google.com/pricing

Firebase. (s. f.-b). *Default bucket and billing requirements for Cloud Storage for Firebase after September 2024*. https://firebase.google.com/docs/storage/faqs-storage-changes-announced-sept-2024

Firebase. (s. f.-c). *Verify ID tokens*. https://firebase.google.com/docs/auth/admin/verify-id-tokens

Firebase. (s. f.-d). *Get started with Cloud Firestore Security Rules*. https://firebase.google.com/docs/firestore/security/get-started

Franks, R., Butcher, N., & Hairston, L. (2026, 28 de julio). *Celebrating 5 years of Jetpack Compose*. Android Developers Blog. https://android-developers.googleblog.com/2026/07/five-years-of-jetpack-compose.html

Frenken, K., & Schor, J. (2017). Putting the sharing economy into perspective. *Environmental Innovation and Societal Transitions, 23*, 3–10. https://doi.org/10.1016/j.eist.2017.01.003

Goldmacher, S. (2021, 3 de abril). How Trump steered supporters into unwitting donations. *The New York Times*. https://www.nytimes.com/2021/04/03/us/politics/trump-donations.html

Google for Developers. (s. f.-a). *Maps SDK for Android usage and billing*. https://developers.google.com/maps/documentation/android-sdk/usage-and-billing

Google for Developers. (s. f.-b). *Google Maps Platform core services pricing list*. https://developers.google.com/maps/billing-and-pricing/pricing

Google for Developers. (s. f.-c). *Maps Compose library*. https://developers.google.com/maps/documentation/android-sdk/maps-compose

Goswami, M., & Arputham, R. M. (2026). Connecting users and providers in the construction equipment industry: A conceptual framework motivated by

the ridesharing industry. *Operations and Supply Chain Innovation, 1*(1), Artículo 3. https://doi.org/10.53941/osci.2026.100003

Instituto Nacional de Estadística e Informática. (2026). *Estadísticas de las tecnologías de información y comunicación en los hogares: Trimestre abril-mayo-junio 2026* (Informe Técnico N.° 03). https://www.inei.gob.pe/media/MenuRecursivo/boletines/boletin-tic-iit2026.pdf

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. https://www.iso.org/standard/78176.html

Lawler, R. (2017, 5 de mayo). Caterpillar acquired Yard Club, a marketplace for construction equipment. *TechCrunch*. https://techcrunch.com/2017/05/05/caterpillar-yard-club-acquisition/

Mathur, A., Acar, G., Friedman, M. J., Lucherini, E., Mayer, J., Chetty, M., & Narayanan, A. (2019). Dark patterns at scale: Findings from a crawl of 11K shopping websites. *Proceedings of the ACM on Human-Computer Interaction, 3*(CSCW), Artículo 81. https://doi.org/10.1145/3359183

Ministerio de Justicia y Derechos Humanos. (2024). *Decreto Supremo N.° 016-2024-JUS, que aprueba el Reglamento de la Ley N.° 29733, Ley de Protección de Datos Personales*. https://www.congreso.gob.pe/Docs/DGP/DIDP/files/ds_016-2024-jus.pdf

Naked Security. (2015, 5 de octubre). *LinkedIn set to pay $13 million compensation over its email persistence*. Sophos. https://news.sophos.com/en-us/2015/10/05/linkedin-set-to-pay-13-million-compensation-over-its-email-persistence

OpenAPI Initiative. (2020). *OpenAPI Specification v3.0.3*. https://spec.openapis.org/oas/v3.0.3

OpenJS Foundation. (s. f.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. https://expressjs.com/

OWASP. (s. f.). *OWASP Mobile Application Security Verification Standard (MASVS)*. https://mas.owasp.org/MASVS/

Rathnayake, I., Ochoa, J. J., Gu, N., Rameezdeen, R., Statsenko, L., & Sandhu, S. (2024). Building the future: Fostering circularity through the new dawn of sharing economy practices in the construction industry. En *Proceedings of the 40th Annual ARCOM Conference* (pp. 849–858). Association of Researchers in Construction Management. https://digital.library.adelaide.edu.au/items/908da04e-eb49-46ff-8f21-3bc67a8f67ab

Roque Mayo, R. A., & Minchán Huamán, Y. M. (2025). *Aplicativo móvil para la gestión de citas, en la empresa Mecánica Automotriz Motor Servís Magno E.I.R.L, Cajamarca 2024* [Tesis de licenciatura, Universidad Privada del Norte]. Repositorio Institucional UPN. https://repositorio.upn.edu.pe/backend/api/core/bitstreams/5d4a1d0c-a4d9-4ee3-b550-30c2e3ab9de5/content

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/scrum-guide.html

Square. (s. f.). *Retrofit*. https://square.github.io/retrofit/

StatCounter. (2026). *Mobile operating system market share Peru*. https://gs.statcounter.com/os-market-share/mobile/peru

Tan, G. (2026, 23 de enero). *Congratulations to EquipmentShare on going public*. Y Combinator. https://www.ycombinator.com/blog/congratulations-to-equipmentshare

ter Huurne, M., Ronteltap, A., Corten, R., & Buskens, V. (2017). Antecedents of trust in the sharing economy: A systematic review. *Journal of Consumer Behaviour, 16*(6), 485–498. https://doi.org/10.1002/cb.1667

United Nations. (s. f.). *Goal 12: Ensure sustainable consumption and production patterns*. https://sdgs.un.org/goals/goal12

W3C. (2024). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

XI. ANEXOS

Anexo 1. Investigación 1: marketing irresponsable en aplicaciones y propuestas éticas

Tabla 15

Casos documentados de marketing irresponsable

Caso	Práctica cuestionada	Consecuencia documentada	Tipo de patrón
Amazon Prime (Unión Europea, 2022)	Cancelación con menús complicados, redacción engañosa, opciones confusas e intentos reiterados de retener al usuario	Tras la acción de la Comisión Europea y las autoridades nacionales, Amazon se comprometió a permitir cancelar en dos pasos con un botón claro (European Commission, 2022)	Obstrucción («roach motel»)
Sitios de reserva de hoteles: Booking.com, Expedia, Agoda, Hotels.com, ebookers y trivago (Reino Unido, 2019)	Falsa impresión de disponibilidad o popularidad, descuentos engañosos y cargos ocultos	Compromisos ante la CMA para eliminar esas prácticas desde septiembre de 2019, sin declaración formal de infracción (Competition and Markets Authority, 2019)	Urgencia y escasez falsas
ebookers.com, caso C-112/11 (Tribunal de Justicia de la UE, 2012)	Seguro de cancelación incluido por defecto en el precio del pasaje	El tribunal estableció que los suplementos opcionales deben aceptarse expresamente (opt-in) (European Parliamentary Research Service, 2012)	Casilla premarcada
LinkedIn, «Add Connections» (EE. UU., 2015)	Invitaciones y dos recordatorios automáticos a los contactos importados del correo del usuario	Acuerdo de 13 millones de dólares en la demanda colectiva Perkins v. LinkedIn (Naked Security, 2015)	Spam a contactos
Campaña de Donald Trump vía WinRed (EE. UU., 2020)	Casilla premarcada de donación semanal recurrente con texto cada vez menos visible	La campaña, el Comité Nacional Republicano y sus cuentas compartidas emitieron más de 530 000 reembolsos por 64,3 millones de dólares en los últimos dos meses y medio de 2020 (Goldmacher, 2021)	Pregunta engañosa y casilla premarcada

Nota. Elaboración propia.

Los cinco casos priorizan una métrica de corto plazo (conversiones, retención o ingresos) sobre la confianza del usuario y terminaron en compromisos regulatorios, acuerdos judiciales o devoluciones masivas. Para una plataforma B2B el riesgo es mayor, porque las decisiones de contratación se consultan entre pares y una empresa que se siente engañada puede advertir a su red de contactos; además, la confianza es un antecedente central de la adopción

en la economía colaborativa (ter Huurne et al., 2017). La Tabla 16 relaciona los principios éticos adoptados con su implementación verificable en el código.

Tabla 16
Principios de diseño ético e implementación en EQUISHARE

Principio	Evita	Implementación en la app
Cancelación simétrica	Obstrucción (Amazon Prime)	Eliminar un equipo o cancelar una reserva toma un botón y una confirmación; el costo de cancelar se informa antes.
Disponibilidad real	Urgencia falsa (sitios de reserva)	Solo se muestran fechas ocupadas por reservas reales; no hay mensajes del tipo «¡solo queda uno!».
Nada preseleccionado	Casillas premarcadas (ebookers, WinRed)	La casilla de privacidad inicia sin marcar; no existen seguros ni servicios añadidos por defecto.
Permisos explícitos	Spam a contactos (LinkedIn)	No se solicita acceso a contactos; la ubicación se pide en contexto, es opcional y no se guarda.
Costos en lenguaje simple	Cargos ocultos	El resumen de la reserva muestra días, precio, total, comisión del 12 % y penalidad del 20 % antes de confirmar.
Transparencia del algoritmo	Decisiones automatizadas opacas	Cada foto rechazada informa el motivo exacto (resolución, formato, tamaño o proporción).

Nota. Elaboración propia.

Anexo 2. Historias de usuario con criterios de aceptación

Tabla 17
Criterios de aceptación de las historias terminadas

Historia	Criterios de aceptación (Dado / Cuando / Entonces)
HU-01 Registro y RUC	• Dado un RUC con dígito verificador incorrecto, cuando la empresa pulsa «Verificar y continuar», entonces se muestra el error bajo el campo y no se envía. • Dado un RUC válido no registrado, entonces la empresa queda verificada y accede a la app. • Dado un RUC ya usado por otra empresa, entonces la API responde 409.
HU-02 Publicar equipo	• Dado un formulario incompleto, cuando se pulsa «Publicar», entonces cada campo inválido muestra su mensaje. • Dado un equipo con foto aprobada y visible activado, entonces aparece en el catálogo. • Dado un equipo sin foto aprobada, entonces se guarda como «Falta foto aprobada» y no se publica.
HU-03 Validación de foto	• Dada una imagen menor a 640×480, mayor a 5 MB, con proporción mayor a 2:1 o que no es JPG, PNG o WebP, entonces se rechaza con el motivo exacto. • Dada una imagen válida, entonces se aprueba y se obtiene su identificador.

Historia	Criterios de aceptación (Dado / Cuando / Entonces)
HU-04 Buscar	• Cuando se escribe en la búsqueda, entonces la lista se actualiza tras 400 ms sin escribir. • Cuando se elige una categoría, entonces solo se muestran equipos de esa categoría y la elección se recuerda. • Cuando se activa «Cerca de mí» con permiso, entonces los equipos se ordenan por distancia dentro de 15 km.
HU-05 Disponibilidad	• Dado un equipo con reservas activas, entonces el detalle lista los rangos ocupados y el calendario los deshabilita.
HU-06 Reservar	• Dado un inicio con menos de 24 h respecto a las 08:00, entonces se informa la regla RN-07. • Dado un rango válido, entonces se muestra el resumen de costos antes de confirmar. • Cuando se confirma, entonces la reserva queda «Pendiente» para la arrendadora.
HU-08 Calificar	• Dada una reserva devuelta hace menos de 48 h y sin reseña propia, entonces se muestra «Calificar». • Cuando se envían de 1 a 5 estrellas y un comentario de 3 a 300 caracteres, entonces se registra y se actualiza el promedio. • Dada una segunda reseña de la misma empresa, entonces la API responde 409.
HU-11 Revisión automática	• Dado un equipo con 5 reseñas y promedio menor a 3,0, entonces pasa a «En revisión» y sale del catálogo.
HU-12 Penalidad	• Dada una cancelación a menos de 12 h de la entrega, entonces se aplica el 20 % y el aviso previo muestra el monto. • Dada una cancelación anticipada, entonces la penalidad es 0.
HU-13 Mapa	• Dada una clave de Maps configurada, entonces la pestaña Mapa muestra un marcador por equipo y abre el detalle desde la ventana de información. • Dado que se niega el permiso, entonces el mapa funciona sin mostrar la posición del usuario.
HU-14 Sin conexión	• Dado que no hay red, entonces se muestra el aviso «Sin conexión» y los datos guardados. • Cuando vuelve la red, entonces el catálogo se actualiza sin intervención del usuario.
HU-15 Editar y eliminar	• Cuando la dueña edita el precio, entonces el catálogo muestra el nuevo valor. • Dado un equipo con reservas activas, cuando se intenta eliminar, entonces la API responde 409 con la explicación. • Dado otro usuario, cuando intenta editar el equipo, entonces la API responde 403.

Nota. Elaboración propia.